

Módulo 6 - Guía práctica

Curso Spring Boot 2026

Jaime Gallo

Índice

Módulo 6 - Guía práctica: de Spring Security a una API JWT stateless	10
Qué vas a construir	10
Requisitos	10
Forma de trabajo	10
Práctica 6.1 - Introducción a Spring Security	11
Paso 6.1.1 - Añadir la dependencia de Spring Security	11
Objetivo	11
Procedimiento	11
Criterio de cierre	11
Paso 6.1.2 - Arrancar y ver el “susto”	11
Objetivo	11
Procedimiento	11
Criterio de cierre	12
Paso 6.1.3 - Autenticarse con la contraseña generada	12
Objetivo	12
Procedimiento	12
Criterio de cierre	12
Paso 6.1.4 - Ver los filtros en acción	12
Objetivo	12
Procedimiento	12
Criterio de cierre	12
Paso 6.1.5 - Crear SecurityConfig	13
Objetivo	13
Procedimiento	13
Criterio de cierre	13
Paso 6.1.6 - Ver el comportamiento sin usuarios definidos por nosotros	13
Objetivo	13
Procedimiento	13
Criterio de cierre	13
Paso 6.1.7 - Permitir todos los endpoints temporalmente	13
Objetivo	13
Procedimiento	14
Criterio de cierre	14
Paso 6.1.8 - Configurar reglas específicas por endpoint	14
Objetivo	14
Procedimiento	14
Criterio de cierre	14
Paso 6.1.9 - Ver la diferencia entre 401 y 403	14
Objetivo	14
Procedimiento	14
Criterio de cierre	15

Paso 6.1.10 - Depurar con logs	15
Objetivo	15
Procedimiento	15
Criterio de cierre	15
Paso 6.1.11 - Errores comunes del ejercicio	15
Objetivo	15
Procedimiento	15
Criterio de cierre	15
Paso 6.1.12 - Reto resuelto: endpoint público sin autenticación	16
Objetivo	16
Procedimiento	16
6.2 — Autenticación con usuarios en memoria	16
Criterio de cierre	16
Práctica 6.2 - Usuarios en memoria, roles y PasswordEncoder	16
Paso 6.2.1 - Repasar el estado actual	16
Objetivo	16
Procedimiento	17
Criterio de cierre	17
Paso 6.2.2 - Añadir un endpoint que devuelva el perfil del usuario	17
Objetivo	17
Procedimiento	17
Criterio de cierre	18
Paso 6.2.3 - Definir el PasswordEncoder	18
Objetivo	18
Procedimiento	18
Criterio de cierre	19
Paso 6.2.4 - Definir el UserDetailsService	19
Objetivo	19
Procedimiento	19
Criterio de cierre	20
Paso 6.2.5 - Cerrar los endpoints	20
Objetivo	20
Procedimiento	20
Criterio de cierre	20
Paso 6.2.6 - Arrancar y probar sin autenticación	21
Objetivo	21
Procedimiento	21
Criterio de cierre	21
Paso 6.2.7 - Autenticarse con el usuario ana	21
Objetivo	21
Procedimiento	21
Criterio de cierre	22
Paso 6.2.8 - Autenticarse con el usuario admin	22
Objetivo	22
Procedimiento	22
Criterio de cierre	22
Paso 6.2.9 - Probar la autorización por rol	22
Objetivo	22
Procedimiento	22
Criterio de cierre	23
Paso 6.2.10 - Probar con credenciales incorrectas	23
Objetivo	23
Procedimiento	23
Criterio de cierre	23

Paso 6.2.11 - Errores comunes del ejercicio	23
Objetivo	23
Procedimiento	23
Criterio de cierre	24
Paso 6.2.12 - Reto resuelto: tercer usuario con roles combinados	24
Objetivo	24
Procedimiento	24
1. Evolucionar InMemoryUserConfig	24
2. Crear el endpoint	24
3. Autorizar cualquiera de los dos roles	25
4. Probar	25
Resultado esperado global de 6.2	25
Restauraciones/evoluciones registradas	25
6.3 — Usuarios en base de datos	25
Criterio de cierre	25
Práctica 6.3 - Usuarios en base de datos y registro	26
Paso 6.3.1 - Crear el paquete auth	26
Objetivo	26
Procedimiento	26
Criterio de cierre	26
Paso 6.3.2 - Crear la entidad Rol	26
Objetivo	26
Procedimiento	26
Criterio de cierre	27
Paso 6.3.3 - Crear la entidad Usuario	27
Objetivo	27
Procedimiento	27
Criterio de cierre	28
Paso 6.3.4 - Crear los repositorios	28
Objetivo	28
Procedimiento	28
Criterio de cierre	28
Paso 6.3.5 - Crear el UserDetailsService personalizado	28
Objetivo	28
Procedimiento	29
Criterio de cierre	29
Paso 6.3.6 - Eliminar los usuarios en memoria	29
Objetivo	29
Procedimiento	29
Criterio de cierre	30
Paso 6.3.7 - Crear el inicializador de usuarios	30
Objetivo	30
Procedimiento	30
Criterio de cierre	30
Paso 6.3.8 - Arrancar y verificar	31
Objetivo	31
Procedimiento	31
Criterio de cierre	31
Paso 6.3.9 - Probar la autenticación	31
Objetivo	31
Procedimiento	31
Criterio de cierre	32
Paso 6.3.10 - Crear los DTOs de registro	32
Objetivo	32

Procedimiento	32
Criterio de cierre	32
Paso 6.3.11 - Crear AuthService y AuthController	32
Objetivo	32
Procedimiento	32
Criterio de cierre	33
Paso 6.3.12 - Configurar el endpoint público y probar	33
Objetivo	33
Procedimiento	33
Criterio de cierre	34
Paso 6.3.13 - Errores comunes del ejercicio	34
Objetivo	34
Procedimiento	34
Criterio de cierre	34
Paso 6.3.14 - Reto resuelto: cambiar la contraseña	34
Objetivo	34
Procedimiento	35
DTO	35
Servicio	35
Controlador	35
Prueba	36
Decisión técnica	36
Resultado esperado global 6.3	36
6.4 — Autorización por roles	36
Criterio de cierre	36
Práctica 6.4 - Autorización por URL y por método	37
Paso 6.4.1 - Repasar el estado actual	37
Objetivo	37
Procedimiento	37
Criterio de cierre	37
Paso 6.4.2 - Activar la seguridad por método	37
Objetivo	37
Procedimiento	37
Criterio de cierre	38
Paso 6.4.3 - Crear un UsuarioController de administración	38
Objetivo	38
Procedimiento	38
Criterio de cierre	38
Paso 6.4.4 - Crear el UsuarioService	39
Objetivo	39
Procedimiento	39
Criterio de cierre	39
Paso 6.4.5 - Añadir reglas de URL y de método combinadas	39
Objetivo	39
Procedimiento	39
Criterio de cierre	40
Paso 6.4.6 - Probar la autorización con distintos usuarios	40
Objetivo	40
Procedimiento	40
Criterio de cierre	40
Paso 6.4.7 - Crear un endpoint para consultar el propio perfil	40
Objetivo	40
Procedimiento	41
Criterio de cierre	41

Paso 6.4.8 - Crear UsuarioPrincipal para 6.7	41
Objetivo	41
Procedimiento	41
Criterio de cierre	42
Paso 6.4.9 - Probar la expresión de autorización	42
Objetivo	42
Procedimiento	42
Criterio de cierre	42
Paso 6.4.10 - Escribir tests de autorización	42
Objetivo	42
Procedimiento	42
Criterio de cierre	43
Paso 6.4.11 - Errores comunes del ejercicio	43
Objetivo	43
Procedimiento	44
Criterio de cierre	44
Paso 6.4.12 - Reto resuelto: endpoint de cambio de rol	44
Objetivo	44
Procedimiento	44
DTO	44
Servicio	44
Controlador	45
Prueba	45
Resultado esperado global 6.4	45
6.5 — Introducción a JWT	46
Criterio de cierre	46
Práctica 6.5 - JWT: estructura, firma y ciclo de vida	46
Paso 6.5.1 - Entender la estructura con un ejemplo	46
Objetivo	46
Procedimiento	46
Criterio de cierre	47
Paso 6.5.2 - Crear una clase para generar un JWT manualmente	47
Objetivo	47
Procedimiento	47
Criterio de cierre	47
Paso 6.5.3 - Decodificar el token generado	47
Objetivo	47
Procedimiento	48
Criterio de cierre	48
Paso 6.5.4 - Modificar el payload sin actualizar la firma	48
Objetivo	48
Procedimiento	48
Criterio de cierre	48
Paso 6.5.5 - Verificar el token con Java	49
Objetivo	49
Procedimiento	49
Criterio de cierre	49
Paso 6.5.6 - Probar con distintos payloads	49
Objetivo	49
Procedimiento	49
Criterio de cierre	50
Paso 6.5.7 - Reflexionar sobre la seguridad	50
Objetivo	50
Procedimiento	50

Criterio de cierre	50
Paso 6.5.8 - Comparar con un token real	50
Objetivo	50
Procedimiento	50
Criterio de cierre	51
Paso 6.5.9 - Errores comunes del ejercicio	51
Objetivo	51
Procedimiento	51
Criterio de cierre	51
Paso 6.5.10 - Experimentar con la expiración	51
Objetivo	51
Procedimiento	51
Criterio de cierre	52
Paso 6.5.11 - Verificar el token con una clave incorrecta	52
Objetivo	52
Procedimiento	52
Criterio de cierre	52
Paso 6.5.12 - Reto resuelto: decodificar un token sin verificar la firma	52
Objetivo	52
Procedimiento	52
Resultado esperado global 6.5	53
6.6 — Generación de tokens JWT	53
Criterio de cierre	53
Práctica 6.6 - Generación y validación de JWT con JJWT	53
Paso 6.6.1 - Añadir las dependencias de JJWT	53
Objetivo	53
Procedimiento	53
Criterio de cierre	54
Paso 6.6.2 - Configurar las propiedades de JWT	54
Objetivo	54
Procedimiento	54
Criterio de cierre	54
Paso 6.6.3 - Crear la configuración de la clave	54
Objetivo	54
Procedimiento	54
Criterio de cierre	55
Paso 6.6.4 - Crear el JwtService	55
Objetivo	55
Procedimiento	55
Criterio de cierre	55
Paso 6.6.5 - Crear los DTOs de login	55
Objetivo	55
Procedimiento	56
Criterio de cierre	56
Paso 6.6.6 - Ampliar AuthService con login y refresh	56
Objetivo	56
Procedimiento	56
Referencia de implementación	56
Criterio de cierre	58
Paso 6.6.7 - Ampliar AuthController	58
Objetivo	58
Procedimiento	58
Criterio de cierre	58
Paso 6.6.8 - Configurar la seguridad para permitir login	59

Objetivo	59
Procedimiento	59
Criterio de cierre	59
Paso 6.6.9 - Arrancar y probar el login	59
Objetivo	59
Procedimiento	59
Criterio de cierre	59
Paso 6.6.10 - Decodificar el token	59
Objetivo	59
Procedimiento	60
Criterio de cierre	60
Paso 6.6.11 - Probar el refresh	60
Objetivo	60
Procedimiento	60
Criterio de cierre	60
Paso 6.6.12 - Errores comunes del ejercicio	60
Objetivo	60
Procedimiento	60
Materialización exacta del contrato de error en el proyecto final	61
Criterio de cierre	61
Paso 6.6.13 - Reto resuelto: endpoint para logout	61
Objetivo	61
Procedimiento	61
Cierre 6.6	61
6.7 — Filtro JWT	62
Criterio de cierre	62
Práctica 6.7 - Filtro JWT y reconstrucción de la identidad	62
Paso 6.7.1 - Verificar JwtService y UsuarioPrincipal	62
Objetivo	62
Procedimiento	62
Criterio de cierre	62
Paso 6.7.2 - Crear JwtAuthenticationFilter	62
Objetivo	62
Procedimiento	62
Referencia de implementación	63
Criterio de cierre	64
Paso 6.7.3 - Registrar el filtro en la cadena	64
Objetivo	64
Procedimiento	64
Criterio de cierre	65
Paso 6.7.4 - Desactivar HTTP Basic	65
Objetivo	65
Procedimiento	65
Criterio de cierre	65
Paso 6.7.5 - Arrancar y probar	65
Objetivo	65
Procedimiento	65
Criterio de cierre	65
Paso 6.7.6 - Verificar UsuarioPrincipal	66
Objetivo	66
Procedimiento	66
Criterio de cierre	66
Paso 6.7.7 - Probar con rol ADMIN	66
Objetivo	66

Procedimiento	66
Criterio de cierre	66
Paso 6.7.8 - Probar con un token inválido	66
Objetivo	66
Procedimiento	66
Criterio de cierre	67
Paso 6.7.9 - Probar con cabecera mal formada	67
Objetivo	67
Procedimiento	67
Criterio de cierre	67
Paso 6.7.10 - Depurar el filtro	67
Objetivo	67
Procedimiento	67
Criterio de cierre	67
Paso 6.7.11 - Errores comunes del ejercicio	67
Objetivo	67
Procedimiento	68
Criterio de cierre	68
Paso 6.7.12 - Reto resuelto: SpEL con ID del usuario	68
Objetivo	68
Procedimiento	68
Cierre 6.7	68
6.8 — Protección de endpoints con JWT	68
Criterio de cierre	68
Práctica 6.8 - Configuración final stateless y manejo de errores	69
Paso 6.8.1 - Crear JwtAuthenticationEntryPoint	69
Objetivo	69
Procedimiento	69
Referencia de implementación	69
Criterio de cierre	70
Paso 6.8.2 - Crear JwtAccessDeniedHandler	70
Objetivo	70
Procedimiento	70
Referencia de implementación	70
Criterio de cierre	71
Paso 6.8.3 - Actualizar SecurityConfig	71
Objetivo	71
Procedimiento	71
Criterio de cierre	71
Paso 6.8.4 - Añadir endpoint de gestor	71
Objetivo	71
Procedimiento	71
Criterio de cierre	71
Paso 6.8.5 - Añadir rol GESTOR al inicializador	72
Objetivo	72
Procedimiento	72
Criterio de cierre	72
Paso 6.8.6 - Probar el flujo completo	72
Objetivo	72
Procedimiento	72
Criterio de cierre	72
Paso 6.8.7 - Probar sin token	72
Objetivo	72
Procedimiento	72

Criterio de cierre	73
Paso 6.8.8 - Probar refresh token	73
Objetivo	73
Procedimiento	73
Criterio de cierre	73
Paso 6.8.9 - Probar logout	73
Objetivo	73
Procedimiento	73
Criterio de cierre	73
Paso 6.8.10 - Escribir tests de integración	73
Objetivo	73
Procedimiento	74
Criterio de cierre	74
Paso 6.8.11 - Errores comunes del ejercicio	74
Objetivo	74
Procedimiento	74
Criterio de cierre	74
Paso 6.8.12 - Reto resuelto: consultar el propio usuario	74
Objetivo	74
Procedimiento	74
Cierre 6.8	75
6.9 — Testing de seguridad	75
Criterio de cierre	75
Práctica 6.9 - Pruebas de seguridad	75
Paso 6.9.1 - Añadir spring-security-test	75
Objetivo	75
Procedimiento	75
Criterio de cierre	75
Paso 6.9.2 - Crear test del UsuarioController	75
Objetivo	75
Procedimiento	76
Criterio de cierre	76
Paso 6.9.3 - Ejecutar los tests	76
Objetivo	76
Procedimiento	76
Criterio de cierre	77
Paso 6.9.4 - Testear endpoint de gestor	77
Objetivo	77
Procedimiento	77
Criterio de cierre	77
Paso 6.9.5 - Testear endpoint público	77
Objetivo	77
Procedimiento	77
Criterio de cierre	77
Paso 6.9.6 - Testear login	77
Objetivo	77
Procedimiento	78
Criterio de cierre	78
Paso 6.9.7 - Testear refresh	78
Objetivo	78
Procedimiento	78
Criterio de cierre	78
Paso 6.9.8 - Testear logout	78
Objetivo	78

Procedimiento	78
Criterio de cierre	78
Paso 6.9.9 - Testear con post-processors	79
Objetivo	79
Procedimiento	79
Criterio de cierre	79
Paso 6.9.10 - Testear con @WithUserDetails	79
Objetivo	79
Procedimiento	79
Criterio de cierre	79
Paso 6.9.11 - Testear el filtro JWT	79
Objetivo	79
Procedimiento	80
Referencia de implementación	80
Criterio de cierre	81
Paso 6.9.12 - Errores comunes del ejercicio	81
Objetivo	81
Procedimiento	81
Criterio de cierre	81
Paso 6.9.13 - Reto resuelto: test completo de autorización	81
Objetivo	81
Procedimiento	82
Resultado esperado global del módulo	82
Criterio de cierre	82

Verificación final del módulo

82

Módulo 6 - Guía práctica: de Spring Security a una API JWT stateless

Qué vas a construir

Partirás del proyecto final de M5 y añadirás seguridad de forma incremental. Primero observarás la protección automática de Spring Security y utilizarás HTTP Basic como mecanismo temporal. Después crearás usuarios y roles, los trasladarás a base de datos, aplicarás autorización por ruta y por método, estudiarás JWT y construirás el flujo completo de login, refresh, filtro Bearer, logout/revocación y pruebas.

Cada paso deja un resultado comprobable. Cuando un experimento es temporal, el propio itinerario indica cuándo retirarlo. El estado final de M6/proyecto/ representa una API STATELESS protegida con JWT y no conserva configuraciones didácticas intermedias.

Requisitos

- Java 17.
- El Maven Wrapper incluido en el proyecto (mvnw / mvnw.cmd).
- El proyecto final de M5.
- curl o una herramienta equivalente para las comprobaciones HTTP.

Forma de trabajo

Ejecuta los pasos en orden. Tras cada cambio compila o realiza la petición indicada y no continúes hasta entender el observable. Los comandos se muestran con sintaxis Unix cuando es más legible; en Windows utiliza mvnw.cmd y adapta las continuaciones de línea cuando sea necesario.

Práctica 6.1 - Introducción a Spring Security

El objetivo es observar primero el comportamiento seguro por defecto y terminar con una configuración explícita. Los estados con HTTP Basic o permitAll son temporales y se cierran dentro del propio punto.

Paso 6.1.1 - Añadir la dependencia de Spring Security

Objetivo

Completar **añadir la dependencia de spring security** y comprobar su efecto antes de continuar. El objetivo es observar primero el comportamiento seguro por defecto y terminar con una configuración explícita. Los estados con HTTP Basic o permitAll son temporales y se cierran dentro del propio punto.

Procedimiento

Qué hacemos. Partimos del pom.xml real de M5 y añadimos spring-boot-starter-security sin retirar ninguna dependencia heredada.

Por qué. Queremos observar el comportamiento seguro por defecto antes de configurarlo.

Archivo. M6/proyecto/pom.xml.

Comando de verificación cuando el árbol M5 esté completamente materializado:

```
./mvnw -B -DskipTests compile
```

Observable. Maven resuelve Spring Security y el proyecto compila sin perder web, validation, JPA, H2, PostgreSQL ni springdoc.

Error posible. Sustituir el POM de M5 por un POM mínimo. **Corrección:** comparar dependencias/plugins con el proyecto de partida; M6 sólo añade/evoluciona.

Criterio de cierre

Antes de continuar, debes poder explicar por qué una petición anónima cambia de comportamiento al añadir el starter y dónde se expresa la política HTTP.

Paso 6.1.2 - Arrancar y ver el “susto”

Objetivo

Completar **arrancar y ver el “susto”** y comprobar su efecto antes de continuar. El objetivo es observar primero el comportamiento seguro por defecto y terminar con una configuración explícita. Los estados con HTTP Basic o permitAll son temporales y se cierran dentro del propio punto.

Procedimiento

```
./mvnw spring-boot:run
```

Observable. Spring Security protege las rutas por defecto y Spring Boot registra una contraseña de desarrollo para el usuario generado. Una petición anónima a un endpoint antes público deja de responder como en M5.

Qué aprendemos. Añadir el starter cambia el comportamiento runtime incluso antes de escribir SecurityConfig.

Criterio de cierre

Antes de continuar, debes poder explicar por qué una petición anónima cambia de comportamiento al añadir el starter y dónde se expresa la política HTTP.

Paso 6.1.3 - Autenticarse con la contraseña generada

Objetivo

Completar **autenticarse con la contraseña generada** y comprobar su efecto antes de continuar. El objetivo es observar primero el comportamiento seguro por defecto y terminar con una configuración explícita. Los estados con HTTP Basic o permitAll son temporales y se cierran dentro del propio punto.

Procedimiento

Toma la contraseña generada del log de **ese arranque** y prueba, por ejemplo:

```
curl -i -u user:<PASSWORD_DEL_LOG> http://localhost:8080/hola
```

Observable. Con credenciales válidas el endpoint puede alcanzarse; sin ellas aparece el rechazo de autenticación. No se copia esa contraseña a properties ni se convierte en configuración permanente.

Criterio de cierre

Antes de continuar, debes poder explicar por qué una petición anónima cambia de comportamiento al añadir el starter y dónde se expresa la política HTTP.

Paso 6.1.4 - Ver los filtros en acción

Objetivo

Completar **ver los filtros en acción** y comprobar su efecto antes de continuar. El objetivo es observar primero el comportamiento seguro por defecto y terminar con una configuración explícita. Los estados con HTTP Basic o permitAll son temporales y se cierran dentro del propio punto.

Procedimiento

Activa temporalmente:

```
logging.level.org.springframework.security=DEBUG
```

Repite una petición anónima y otra autenticada.

Observable. El log permite ver la cadena de seguridad y el tratamiento de la identidad.

Restauración. El nivel DEBUG es una ayuda diagnóstica temporal; no queda forzado como decisión final de producción.

Criterio de cierre

Antes de continuar, debes poder explicar por qué una petición anónima cambia de comportamiento al añadir el starter y dónde se expresa la política HTTP.

Paso 6.1.5 - Crear SecurityConfig

Objetivo

Completar **crear securityconfig** y comprobar su efecto antes de continuar. El objetivo es observar primero el comportamiento seguro por defecto y terminar con una configuración explícita. Los estados con HTTP Basic o permitAll son temporales y se cierran dentro del propio punto.

Procedimiento

El proyecto incluye src/main/java/es/mecd/demo/miproyecto/config/SecurityConfig.java con configuración moderna basada en bean SecurityFilterChain.

Puntos obligatorios del estado 6.1:

```
.csrf(AbstractHttpConfigurer::disable)
.cors(Customizer.withDefaults())
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/v1/public/**").permitAll()
    .anyRequest().authenticated())
.httpBasic(Customizer.withDefaults())
```

Observable. Existe una política explícita; no se usa WebSecurityConfigurerAdapter.

Criterio de cierre

Antes de continuar, debes poder explicar por qué una petición anónima cambia de comportamiento al añadir el starter y dónde se expresa la política HTTP.

Paso 6.1.6 - Ver el comportamiento sin usuarios definidos por nosotros

Objetivo

Completar **ver el comportamiento sin usuarios definidos por nosotros** y comprobar su efecto antes de continuar. El objetivo es observar primero el comportamiento seguro por defecto y terminar con una configuración explícita. Los estados con HTTP Basic o permitAll son temporales y se cierran dentro del propio punto.

Procedimiento

No declares todavía un UserDetailsService propio. Reinicia y comprueba que el usuario de desarrollo autoconfigurado sigue permitiendo observar HTTP Basic.

Objetivo pedagógico. Separar “cadena de seguridad” de “fuente de usuarios”. Los usuarios controlados llegan en 6.2.

Criterio de cierre

Antes de continuar, debes poder explicar por qué una petición anónima cambia de comportamiento al añadir el starter y dónde se expresa la política HTTP.

Paso 6.1.7 - Permitir todos los endpoints temporalmente

Objetivo

Completar **permitir todos los endpoints temporalmente** y comprobar su efecto antes de continuar. El objetivo es observar primero el comportamiento seguro por defecto y terminar con una configuración explícita. Los estados con HTTP Basic o permitAll son temporales y se cierran dentro del propio punto.

Procedimiento

Experimento temporal. Sustituye momentáneamente las reglas por:

```
.anyRequest().permitAll()
```

Arranca y comprueba que /hola y /api/v1/alumnos vuelven a ser accesibles sin autenticación.

No avanzar dejando este estado activo. Este paso existe para demostrar que la política declarativa controla el acceso.

Criterio de cierre

Antes de continuar, debes poder explicar por qué una petición anónima cambia de comportamiento al añadir el starter y dónde se expresa la política HTTP.

Paso 6.1.8 - Configurar reglas específicas por endpoint

Objetivo

Completar **configurar reglas específicas por endpoint** y comprobar su efecto antes de continuar. El objetivo es observar primero el comportamiento seguro por defecto y terminar con una configuración explícita. Los estados con HTTP Basic o permitAll son temporales y se cierran dentro del propio punto.

Procedimiento

Cierre del experimento. Elimina el permitAll() global y deja únicamente rutas públicas explícitas; el resto vuelve a authenticated().

```
.requestMatchers("/api/v1/public/**").permitAll()  
.anyRequest().authenticated()
```

Verificación negativa:

```
curl -i http://localhost:8080/api/v1/alumnos
```

Debe volver a requerir autenticación.

Criterio de cierre

Antes de continuar, debes poder explicar por qué una petición anónima cambia de comportamiento al añadir el starter y dónde se expresa la política HTTP.

Paso 6.1.9 - Ver la diferencia entre 401 y 403

Objetivo

Completar **ver la diferencia entre 401 y 403** y comprobar su efecto antes de continuar. El objetivo es observar primero el comportamiento seguro por defecto y terminar con una configuración explícita. Los estados con HTTP Basic o permitAll son temporales y se cierran dentro del propio punto.

Procedimiento

Para obtener ambos observables de forma controlada, usa temporalmente una regla de rol sobre un endpoint existente:

```
.requestMatchers("/api/v1/alumnos/**").hasRole("ADMIN")
```

- Sin credenciales: se observa el flujo de **401**.

- Con el usuario generado, que no posee ADMIN: se observa **403**.

Restauración del experimento. Después de observar ambos códigos, devuelve `/api/v1/alumnos/**` al nivel previsto para el cierre 6.1 (`authenticated()`); los roles controlados se introducen en 6.2/6.4.

Criterio de cierre

Antes de continuar, debes poder explicar por qué una petición anónima cambia de comportamiento al añadir el starter y dónde se expresa la política HTTP.

Paso 6.1.10 - Depurar con logs

Objetivo

Completar **depurar con logs** y comprobar su efecto antes de continuar. El objetivo es observar primero el comportamiento seguro por defecto y terminar con una configuración explícita. Los estados con HTTP Basic o `permitAll` son temporales y se cierran dentro del propio punto.

Procedimiento

Repite peticiones con DEBUG de Spring Security y relaciona cada código con el punto de la cadena que tomó la decisión. Después restaura el logging ordinario.

Criterio de cierre

Antes de continuar, debes poder explicar por qué una petición anónima cambia de comportamiento al añadir el starter y dónde se expresa la política HTTP.

Paso 6.1.11 - Errores comunes del ejercicio

Objetivo

Completar **errores comunes del ejercicio** y comprobar su efecto antes de continuar. El objetivo es observar primero el comportamiento seguro por defecto y terminar con una configuración explícita. Los estados con HTTP Basic o `permitAll` son temporales y se cierran dentro del propio punto.

Procedimiento

Comprueba explícitamente estos fallos antes de cerrar el punto:

- dejar `anyRequest().permitAll()` activo;
- abrir Swagger u otras rutas por comodidad sin justificarlo;
- desactivar CORS al introducir Security;
- confundir 401 y 403;
- introducir `WebSecurityConfigurerAdapter`;
- esconder una contraseña fija en el repositorio;
- borrar tests heredados porque ahora reciben 401.

La corrección de los tests heredados debe consistir en adaptarlos al nuevo contrato de seguridad, no en eliminarlos.

Criterio de cierre

Antes de continuar, debes poder explicar por qué una petición anónima cambia de comportamiento al añadir el starter y dónde se expresa la política HTTP.

Paso 6.1.12 - Reto resuelto: endpoint público sin autenticación

Objetivo

Completar **reto resuelto: endpoint público sin autenticación** y comprobar su efecto antes de continuar. El objetivo es observar primero el comportamiento seguro por defecto y terminar con una configuración explícita. Los estados con HTTP Basic o permitAll son temporales y se cierran dentro del propio punto.

Procedimiento

El proyecto incorpora PublicInfoController:

```
@RestController
@RequestMapping("/api/v1/public")
public class PublicInfoController {
    @GetMapping("/info")
    public Map<String, String> info() {
        return Map.of("modulo", "M6", "seguridad", "activa");
    }
}
```

Prueba:

```
curl -i http://localhost:8080/api/v1/public/info
curl -i http://localhost:8080/api/v1/alumnos
```

Observable esperado. El primero es público; el segundo continúa protegido.

Cierre 6.1. No queda permitAll() global; HTTP Basic y el usuario generado son estados de aprendizaje que evolucionarán en los puntos siguientes, no la arquitectura final M6.

6.2 — Autenticación con usuarios en memoria

Estado temporal controlado. Este punto introduce InMemoryUserDetailsManager y mantiene HTTP Basic para poder observar el flujo de autenticación. Los usuarios en memoria se eliminan expresamente en 6.3.6; HTTP Basic se cierra en 6.7.4. No representan el diseño final JWT de M6.

Criterio de cierre

Antes de continuar, debes poder explicar por qué una petición anónima cambia de comportamiento al añadir el starter y dónde se expresa la política HTTP.

Práctica 6.2 - Usuarios en memoria, roles y PasswordEncoder

Aquí aislamos el contrato de Spring Security con usuarios controlados en memoria. Esto permite entender roles, authorities y codificación de contraseñas antes de introducir persistencia.

Paso 6.2.1 - Repasar el estado actual

Objetivo

Completar **repasar el estado actual** y comprobar su efecto antes de continuar. Aquí aislamos el contrato de Spring Security con usuarios controlados en memoria. Esto permite entender roles, authorities y codificación de contraseñas antes de introducir persistencia.

Procedimiento

Qué hacemos. Antes de añadir usuarios propios, verificamos el cierre de 6.1:

- spring-boot-starter-security está presente;
- existe un SecurityFilterChain;
- /api/v1/public/** es público;
- el resto requiere autenticación;
- el permitAll() global experimental ya no existe;
- CORS M5 continúa configurado.

Por qué. 6.2 debe evolucionar 6.1, no abrir una implementación paralela.

Archivos.

- src/main/java/es/meecd/demo/miproyecto/config/SecurityConfig.java
- src/main/java/es/meecd/demo/miproyecto/config/CorsConfig.java
- pom.xml

Verificación cuando el árbol M5 esté materializado:

```
./mvnw -B test
```

Observable. El proyecto mantiene el contrato acumulado; todavía no hay usuarios controlados por el curso.

Pregunta. ¿Qué diferencia hay entre definir una política de autorización y definir de dónde salen los usuarios?

Criterio de cierre

Al cerrar el punto, ana, admin y gestor deben producir authorities distintas y las contraseñas nunca deben almacenarse en texto plano.

Paso 6.2.2 - Añadir un endpoint que devuelva el perfil del usuario

Objetivo

Completar **añadir un endpoint que devuelva el perfil del usuario** y comprobar su efecto antes de continuar. Aquí aislamos el contrato de Spring Security con usuarios controlados en memoria. Esto permite entender roles, authorities y codificación de contraseñas antes de introducir persistencia.

Procedimiento

Creamos:

```
src/main/java/es/meecd/demo/miproyecto/common/controller/PerfilController.java
```

```
package es.meecd.demo.miproyecto.common.controller;
```

```
import java.util.Map;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
@RequestMapping("/api/v1/perfil")
public class PerfilController {
```

```

@GetMapping
public Map<String, Object> perfil(Authentication authentication) {
    return Map.of(
        "usuario", authentication.getName(),
        "authorities", authentication.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority)
            .toList());
}
}

```

Qué demuestra. Spring puede inyectar el Authentication ya resuelto por la cadena. El controlador no vuelve a validar la contraseña.

Comando posterior:

```
curl -i -u ana:ana123 http://localhost:8080/api/v1/perfil
```

Observable esperado una vez completados los pasos siguientes.

```

{
  "usuario": "ana",
  "authorities": ["ROLE_USER"]
}

```

Error posible. authentication nulo porque la ruta se dejó pública accidentalmente. **Corrección.** Mantener /api/v1/perfil dentro de anyRequest().authenticated().

Criterio de cierre

Al cerrar el punto, ana, admin y gestor deben producir authorities distintas y las contraseñas nunca deben almacenarse en texto plano.

Paso 6.2.3 - Definir el PasswordEncoder

Objetivo

Completar **definir el passwordencoder** y comprobar su efecto antes de continuar. Aquí aislamos el contrato de Spring Security con usuarios controlados en memoria. Esto permite entender roles, authorities y codificación de contraseñas antes de introducir persistencia.

Procedimiento

El encoder es un bean estable; no pertenece sólo al estado temporal de usuarios en memoria porque 6.3 también lo necesitará.

En SecurityConfig:

```

@Bean
PasswordEncoder passwordEncoder() {
    return PasswordEncoderFactories.createDelegatingPasswordEncoder();
}

```

Por qué no guardamos ana123 directamente. La contraseña que conoce el usuario es una entrada; el sistema conserva/compara una representación codificada.

Verificación conceptual:

```

String hash1 = passwordEncoder.encode("ana123");
String hash2 = passwordEncoder.encode("ana123");

```

```
assert !hash1.equals(hash2);
assert passwordEncoder.matches("ana123", hash1);
assert passwordEncoder.matches("ana123", hash2);
```

Que dos hashes sean diferentes no significa que una contraseña sea incorrecta: BCrypt incorpora salt.

Error posible. NoSuchBeanDefinitionException: PasswordEncoder. **Corrección.** Comprobar que el método está anotado con @Bean y que su clase está bajo el escaneo de Spring.

Pregunta. ¿Por qué matches funciona aunque dos llamadas a encode puedan producir hashes diferentes?

Criterio de cierre

Al cerrar el punto, ana, admin y gestor deben producir authorities distintas y las contraseñas nunca deben almacenarse en texto plano.

Paso 6.2.4 - Definir el UserDetailsService

Objetivo

Completar **definir el userdetailsservice** y comprobar su efecto antes de continuar. Aquí aislamos el contrato de Spring Security con usuarios controlados en memoria. Esto permite entender roles, authorities y codificación de contraseñas antes de introducir persistencia.

Procedimiento

Creamos una configuración explícitamente temporal:

src/main/java/es/meecd/demo/miproyecto/config/InMemoryUserConfig.java

```
@Configuration
public class InMemoryUserConfig {

    @Bean
    UserDetailsService userDetailsService(PasswordEncoder passwordEncoder) {
        UserDetails ana = User.builder()
            .username("ana")
            .password(passwordEncoder.encode("ana123"))
            .roles("USER")
            .build();

        UserDetails admin = User.builder()
            .username("admin")
            .password(passwordEncoder.encode("admin123"))
            .roles("ADMIN", "USER")
            .build();

        return new InMemoryUserDetailsManager(ana, admin);
    }
}
```

Transición a usuarios controlados. A partir de aquí dejamos de depender del usuario aleatorio de desarrollo generado por Boot y controlamos las identidades didácticas.

Observable. ana tiene ROLE_USER; admin tiene ROLE_ADMIN y ROLE_USER.

Error posible. Escribir .roles("ROLE_ADMIN"). **Corrección.** Con roles(...) se usa "ADMIN"; Spring añade ROLE_.

Pregunta. ¿Y el usuario ana necesita el rol ADMIN para consultar un recurso ordinario?

Criterio de cierre

Al cerrar el punto, ana, admin y gestor deben producir authorities distintas y las contraseñas nunca deben almacenarse en texto plano.

Paso 6.2.5 - Cerrar los endpoints

Objetivo

Completar **cerrar los endpoints** y comprobar su efecto antes de continuar. Aquí aislamos el contrato de Spring Security con usuarios controlados en memoria. Esto permite entender roles, authorities y codificación de contraseñas antes de introducir persistencia.

Procedimiento

Evolucionamos el SecurityFilterChain:

http

```
.csrf(AbstractHttpConfigurer::disable)
.cors(Customizer.withDefaults())
.headers(headers -> headers.frameOptions(frame -> frame.sameOrigin()))
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/v1/public/**").permitAll()
    .requestMatchers("/h2-console/**").permitAll()
    .requestMatchers(
        "/swagger-ui/**",
        "/swagger-ui.html",
        "/v3/api-docs/**").permitAll()
    .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
    .requestMatchers("/api/v1/gestor/**")
        .hasAnyRole("GESTOR", "ADMIN")
    .anyRequest().authenticated())
.httpBasic(Customizer.withDefaults());
```

- Conservamos cors(Customizer.withDefaults()) porque M5 ya tiene CORS funcional.
- sameOrigin() permite que la consola H2 pueda mostrarse dentro de frame en el estado de desarrollo.
- No añadimos permitAll() global.
- Swagger/OpenAPI sigue accesible para poder inspeccionar el contrato del API durante el curso.

Importante. H2 público es una comodidad del entorno pedagógico. No debe extrapolarse sin más a producción.

Pregunta. ¿Qué pasaría si anyRequest() apareciera antes que las reglas concretas?

Criterio de cierre

Al cerrar el punto, ana, admin y gestor deben producir authorities distintas y las contraseñas nunca deben almacenarse en texto plano.

Paso 6.2.6 - Arrancar y probar sin autenticación

Objetivo

Completar **arrancar y probar sin autenticación** y comprobar su efecto antes de continuar. Aquí aislamos el contrato de Spring Security con usuarios controlados en memoria. Esto permite entender roles, authorities y codificación de contraseñas antes de introducir persistencia.

Procedimiento

Una vez materializado el proyecto completo:

```
./mvnw spring-boot:run
```

En otra terminal:

```
curl -i http://localhost:8080/api/v1/alumnos  
curl -i http://localhost:8080/api/v1/public/info
```

Observables.

- /api/v1/alumnos → 401 Unauthorized y, en el estado Basic, cabecera WWW-Authenticate.
- /api/v1/public/info → 200 OK.

Diagnóstico. Si ambos devuelven 200, una regla demasiado amplia ha abierto el API. Si ambos devuelven 401, la excepción pública no se está aplicando.

Pregunta. ¿Por qué el mismo SecurityFilterChain puede producir 200 y 401 según la URL?

Criterio de cierre

Al cerrar el punto, ana, admin y gestor deben producir authorities distintas y las contraseñas nunca deben almacenarse en texto plano.

Paso 6.2.7 - Autenticarse con el usuario ana

Objetivo

Completar **autenticarse con el usuario ana** y comprobar su efecto antes de continuar. Aquí aislamos el contrato de Spring Security con usuarios controlados en memoria. Esto permite entender roles, authorities y codificación de contraseñas antes de introducir persistencia.

Procedimiento

```
curl -i -u ana:ana123 http://localhost:8080/api/v1/alumnos  
curl -i -u ana:ana123 http://localhost:8080/api/v1/perfil
```

Observable. La consulta ordinaria debe estar autorizada y el perfil debe mostrar:

```
{  
  "usuario": "ana",  
  "authorities": ["ROLE_USER"]  
}
```

El orden de una colección de authorities no debe usarse como contrato si hay varias; comprobaremos contenido, no una ordenación accidental.

Pregunta. ¿Qué authority tiene ana y de dónde proviene el prefijo ROLE_?

Criterio de cierre

Al cerrar el punto, ana, admin y gestor deben producir authorities distintas y las contraseñas nunca deben almacenarse en texto plano.

Paso 6.2.8 - Autenticarse con el usuario admin

Objetivo

Completar **autenticarse con el usuario admin** y comprobar su efecto antes de continuar. Aquí aislamos el contrato de Spring Security con usuarios controlados en memoria. Esto permite entender roles, authorities y codificación de contraseñas antes de introducir persistencia.

Procedimiento

```
curl -i -u admin:admin123 http://localhost:8080/api/v1/perfil
```

Observable. El perfil contiene ROLE_ADMIN y ROLE_USER.

Por qué conserva también USER. En este diseño didáctico ADMIN puede ejercer las capacidades ordinarias de USER además de las administrativas. No es una regla intrínseca de Spring Security; es una decisión de roles del curso.

Pregunta. ¿Sería obligatorio que todo ADMIN tuviera también USER? No: depende del modelo de autorización.

Criterio de cierre

Al cerrar el punto, ana, admin y gestor deben producir authorities distintas y las contraseñas nunca deben almacenarse en texto plano.

Paso 6.2.9 - Probar la autorización por rol

Objetivo

Completar **probar la autorización por rol** y comprobar su efecto antes de continuar. Aquí aislamos el contrato de Spring Security con usuarios controlados en memoria. Esto permite entender roles, authorities y codificación de contraseñas antes de introducir persistencia.

Procedimiento

Primero con un usuario autenticado pero no autorizado:

```
curl -i -u ana:ana123 http://localhost:8080/api/v1/admin/usuarios
```

Observable: 403 Forbidden.

Luego:

```
curl -i -u admin:admin123 http://localhost:8080/api/v1/admin/usuarios
```

En este momento el endpoint administrativo aún no existe. El usuario admin supera la autorización y la resolución del recurso puede terminar en 404 Not Found. Lo importante de este paso es que **ya no es 403**.

Diferencia observable.

- 401: no existe una autenticación válida.
- 403: existe identidad, pero carece del permiso.
- 404: la seguridad permitió continuar, pero no hay recurso/mapping coincidente.

Pregunta. ¿Por qué un 404 después de autenticarse como admin es una señal distinta a un 403?

Criterio de cierre

Al cerrar el punto, ana, admin y gestor deben producir authorities distintas y las contraseñas nunca deben almacenarse en texto plano.

Paso 6.2.10 - Probar con credenciales incorrectas

Objetivo

Completar **probar con credenciales incorrectas** y comprobar su efecto antes de continuar. Aquí aislamos el contrato de Spring Security con usuarios controlados en memoria. Esto permite entender roles, authorities y codificación de contraseñas antes de introducir persistencia.

Procedimiento

```
curl -i -u ana:contraseñaIncorrecta \
  http://localhost:8080/api/v1/alumnos
```

```
curl -i -u pedro:loQueSea \
  http://localhost:8080/api/v1/alumnos
```

Observable: ambos casos producen 401.

No devolvemos “el usuario existe pero la contraseña falla” frente a “el usuario no existe”. Dar respuestas diferentes facilitaría enumerar cuentas válidas.

Pregunta. ¿Qué información podría obtener un atacante si la API distinguiera públicamente ambos casos?

Criterio de cierre

Al cerrar el punto, ana, admin y gestor deben producir authorities distintas y las contraseñas nunca deben almacenarse en texto plano.

Paso 6.2.11 - Errores comunes del ejercicio

Objetivo

Completar **errores comunes del ejercicio** y comprobar su efecto antes de continuar. Aquí aislamos el contrato de Spring Security con usuarios controlados en memoria. Esto permite entender roles, authorities y codificación de contraseñas antes de introducir persistencia.

Procedimiento

Error	Causa probable	Corrección
401 siempre	credenciales mal enviadas o usuario inexistente	revisar <code>curl -u</code> y <code>UserDetailsService</code>
403 con usuario autenticado	no posee el rol necesario	revisar <code>.roles(...)</code> y la regla de URL
la contraseña no coincide	no se codificó con el mismo <code>PasswordEncoder</code>	crearla con <code>passwordEncoder.encode(...)</code>
<code>NoSuchBeanDefinitionException</code>	falta el bean	declarar el <code>@Bean</code> estable
<code>PasswordEncoder</code> ciclo al arrancar	dependencia circular entre configuración/beans	separar responsabilidades e inyección
la aplicación no compila	import equivocado o ausente	usar imports de Spring Security actuales

Error	Causa probable	Corrección
no aparecen logs de seguridad búsqueda de usuario incorrecta	nivel DEBUG no activado UserDetailsService no resuelve el username esperado	activarlo sólo para diagnóstico revisar implementación/datos
CORS preflight falla	Security intercepta OPTIONS sin integrar CORS	conservar <code>.cors(...)</code> y probar preflight
se usa NoOpPasswordEncoder	atajo inseguro	eliminarlo y usar el encoder del curso

Comprobación de cierre. No convertir un error de test en “solución” eliminando el test heredado.

Criterio de cierre

Al cerrar el punto, ana, admin y gestor deben producir authorities distintas y las contraseñas nunca deben almacenarse en texto plano.

Paso 6.2.12 - Reto resuelto: tercer usuario con roles combinados

Objetivo

Completar **reto resuelto: tercer usuario con roles combinados** y comprobar su efecto antes de continuar. Aquí aislamos el contrato de Spring Security con usuarios controlados en memoria. Esto permite entender roles, authorities y codificación de contraseñas antes de introducir persistencia.

Procedimiento

El reto consiste en un usuario gestor con GESTOR y USER, y una ruta accesible por GESTOR o ADMIN.

1. Evolucionar InMemoryUserConfig

```
UserDetails gestor = User.builder()
    .username("gestor")
    .password(passwordEncoder.encode("gestor123"))
    .roles("GESTOR", "USER")
    .build();

return new InMemoryUserDetailsManager(ana, admin, gestor);
```

2. Crear el endpoint

src/main/java/es/meecd/demo/miproyecto/auth/GestorController.java

```
@RestController
@RequestMapping("/api/v1/gestor")
public class GestorController {

    @GetMapping("/documentos")
    public Map<String, String> documentos() {
        return Map.of("mensaje", "Lista de documentos");
    }
}
```

3. Autorizar cualquiera de los dos roles

```
.requestMatchers("/api/v1/gestor/**")  
.hasAnyRole("GESTOR", "ADMIN")
```

4. Probar

```
# Ana (USER) -> 403  
curl -i -u ana:ana123 \  
http://localhost:8080/api/v1/gestor/documentos  
  
# Gestor -> 200  
curl -i -u gestor:gestor123 \  
http://localhost:8080/api/v1/gestor/documentos  
  
# Admin -> 200  
curl -i -u admin:admin123 \  
http://localhost:8080/api/v1/gestor/documentos
```

Pregunta. ¿Qué diferencia existe entre `hasRole("ADMIN")` y `hasAnyRole("GESTOR", "ADMIN")`?

Resultado esperado global de 6.2

Al cerrar el punto deben existir:

- PasswordEncoder basado en DelegatingPasswordEncoder;
- UserDetailsService temporal con ana, admin y gestor;
- reglas de autorización por endpoint;
- PerfilController;
- endpoint de gestor del reto;
- autenticación Basic verificable con curl;
- diferencias 401/403 demostradas;
- CORS heredado de M5 sigue operativo.

Restauraciones/evoluciones registradas

- **T6.2-A — usuarios en memoria:** permanece activo únicamente hasta **6.3.6**, donde se elimina al sustituirlo por UserDetailsService de base de datos.
- **T6.2-B — HTTP Basic:** continúa para observar el mecanismo durante 6.3–6.6 y se **desactiva en 6.7.4** al entrar en funcionamiento el filtro JWT.
- El PasswordEncoder **no se elimina:** se conserva para registrar/verificar contraseñas persistentes.

6.3 — Usuarios en base de datos

Este punto **cierra** los usuarios en memoria de 6.2. HTTP Basic continúa sólo como mecanismo pedagógico de transporte de credenciales hasta que 6.7 lo sustituya por JWT.

Criterio de cierre

Al cerrar el punto, ana, admin y gestor deben producir authorities distintas y las contraseñas nunca deben almacenarse en texto plano.

Práctica 6.3 - Usuarios en base de datos y registro

Este punto sustituye los usuarios temporales por entidades JPA y un UserDetailsService propio. El registro debe crear cuentas seguras sin exponer ni aceptar privilegios arbitrarios.

Paso 6.3.1 - Crear el paquete auth

Objetivo

Completar **crear el paquete auth** y comprobar su efecto antes de continuar. Este punto sustituye los usuarios temporales por entidades JPA y un UserDetailsService propio. El registro debe crear cuentas seguras sin exponer ni aceptar privilegios arbitrarios.

Procedimiento

Qué hacemos. Consolidamos bajo:

src/main/java/es/mecd/demo/miproyecto/auth/

las entidades, repositorios, servicio de identidad, DTOs y operaciones de autenticación.

El GestorController creado en el reto 6.2 ya está en este paquete y se conserva.

Por qué. La organización por funcionalidad sigue el patrón de M5: los elementos de una misma capacidad permanecen juntos.

Observable. El paquete auth es una evolución acumulativa, no un proyecto paralelo.

Criterio de cierre

Al cerrar el punto, la autenticación debe depender de la base de datos; la configuración en memoria ya no debe existir.

Paso 6.3.2 - Crear la entidad Rol

Objetivo

Completar **crear la entidad rol** y comprobar su efecto antes de continuar. Este punto sustituye los usuarios temporales por entidades JPA y un UserDetailsService propio. El registro debe crear cuentas seguras sin exponer ni aceptar privilegios arbitrarios.

Procedimiento

Archivo:

src/main/java/es/mecd/demo/miproyecto/auth/Rol.java

```
@Entity
@Table(name = "roles")
public class Rol {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String nombre;

    @Column(length = 200)
```

```

    private String descripcion;

    @ManyToMany(mappedBy = "roles")
    private Set<Usuario> usuarios = new HashSet<>();

    protected Rol() {
    }

    public Rol(String nombre, String descripcion) {
        this.nombre = nombre;
        this.descripcion = descripcion;
    }

    // getters/setters
}

```

Observable. Rol no define @JoinTable; es el lado inverso.

Error posible. Definir @JoinTable también aquí. **Corrección.** Sólo Usuario. roles es propietario.

Pregunta. ¿Qué ocurriría si ambos lados intentasen crear su propia tabla de unión?

Criterio de cierre

Al cerrar el punto, la autenticación debe depender de la base de datos; la configuración en memoria ya no debe existir.

Paso 6.3.3 - Crear la entidad Usuario

Objetivo

Completar **crear la entidad usuario** y comprobar su efecto antes de continuar. Este punto sustituye los usuarios temporales por entidades JPA y un UserDetailsService propio. El registro debe crear cuentas seguras sin exponer ni aceptar privilegios arbitrarios.

Procedimiento

Archivo:

src/main/java/es/mecd/demo/miproyecto/auth/Usuario.java

Campos:

- id;
- username único;
- password codificada;
- email único;
- activo;
- fechaCreacion;
- roles.

Relación:

```

@ManyToMany(fetch = FetchType.EAGER)
@JoinTable(
    name = "usuarios_roles",
    joinColumns = @JoinColumn(name = "usuario_id"),
    inverseJoinColumns = @JoinColumn(name = "rol_id"))
private Set<Rol> roles = new HashSet<>();

```

Observable. Hibernate podrá construir tres tablas: usuarios, roles y usuarios_roles.

Nota técnica. EAGER se mantiene porque este modelo necesita authorities durante autenticación. En un dominio más pesado evaluaríamos LAZY + consulta explícita.

Pregunta. ¿Por qué Set encaja mejor que List para roles?

Criterio de cierre

Al cerrar el punto, la autenticación debe depender de la base de datos; la configuración en memoria ya no debe existir.

Paso 6.3.4 - Crear los repositorios

Objetivo

Completar **crear los repositorios** y comprobar su efecto antes de continuar. Este punto sustituye los usuarios temporales por entidades JPA y un UserDetailsService propio. El registro debe crear cuentas seguras sin exponer ni aceptar privilegios arbitrarios.

Procedimiento

RolRepository:

```
public interface RolRepository extends JpaRepository<Rol, Long> {
    Optional<Rol> findByNombre(String nombre);
    boolean existsByNombre(String nombre);
}
```

UsuarioRepository:

```
public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
    Optional<Usuario> findByUsername(String username);
    boolean existsByUsername(String username);
    boolean existsByEmail(String email);
}
```

Por qué. Spring Data deriva las consultas a partir de sus nombres y mantiene los detalles SQL fuera de la capa de seguridad.

Observable. Podemos localizar una identidad por el mismo username que recibe Spring Security y validar duplicados antes de guardar.

Criterio de cierre

Al cerrar el punto, la autenticación debe depender de la base de datos; la configuración en memoria ya no debe existir.

Paso 6.3.5 - Crear el UserDetailsService personalizado

Objetivo

Completar **crear el userDetailsservice personalizado** y comprobar su efecto antes de continuar. Este punto sustituye los usuarios temporales por entidades JPA y un UserDetailsService propio. El registro debe crear cuentas seguras sin exponer ni aceptar privilegios arbitrarios.

Procedimiento

Creemos UsuarioDetailsService.

```
@Service
public class UsuarioDetailsService implements UserDetailsService {

    private final UsuarioRepository usuarioRepository;

    public UsuarioDetailsService(UsuarioRepository usuarioRepository) {
        this.usuarioRepository = usuarioRepository;
    }

    @Override
    @Transactional(readOnly = true)
    public UserDetails loadUserByUsername(String username) {
        Usuario usuario = usuarioRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException(
                "Usuario no encontrado: " + username));

        List<GrantedAuthority> authorities = usuario.getRoles().stream()
            .map(rol -> (GrantedAuthority)
                new SimpleGrantedAuthority("ROLE_" + rol.getNombre()))
            .toList();

        return User.builder()
            .username(usuario.getUsername())
            .password(usuario.getPassword())
            .authorities(authorities)
            .accountExpired(false)
            .accountLocked(!usuario.isActivo())
            .credentialsExpired(false)
            .disabled(!usuario.isActivo())
            .build();
    }
}
```

Observable. Un rol persistido como ADMIN se expone a Spring Security como ROLE_ADMIN.

Error posible. Guardar ROLE_ADMIN y volver a prefijarlo. **Corrección.** En nuestra base se guardan nombres sin ROLE_.

Criterio de cierre

Al cerrar el punto, la autenticación debe depender de la base de datos; la configuración en memoria ya no debe existir.

Paso 6.3.6 - Eliminar los usuarios en memoria

Objetivo

Completar **eliminar los usuarios en memoria** y comprobar su efecto antes de continuar. Este punto sustituye los usuarios temporales por entidades JPA y un UserDetailsService propio. El registro debe crear cuentas seguras sin exponer ni aceptar privilegios arbitrarios.

Procedimiento

Cierre de la transición.

Eliminamos físicamente:

config/InMemoryUserConfig.java

No se elimina:

@Bean

PasswordEncoder passwordEncoder()

porque la autenticación persistente y el registro siguen necesitando el encoder.

Comprobación negativa. El proyecto final del paso no puede contener:

InMemoryUserDetailsManager

en código de producción.

Por qué. Dos fuentes de usuario compitiendo harían ambiguo el mecanismo de autenticación y, sobre todo, dejarían vivo un estado pedagógico que 6.3 está diseñado para sustituir.

Criterio de cierre

Al cerrar el punto, la autenticación debe depender de la base de datos; la configuración en memoria ya no debe existir.

Paso 6.3.7 - Crear el inicializador de usuarios

Objetivo

Completar **crear el inicializador de usuarios** y comprobar su efecto antes de continuar. Este punto sustituye los usuarios temporales por entidades JPA y un UserDetailsService propio. El registro debe crear cuentas seguras sin exponer ni aceptar privilegios arbitrarios.

Procedimiento

Creamos UsuariosInicialesConfig.

El ejemplo utiliza un guard global `count() > 0`. Conservamos la explicación, pero el proyecto mejora el algoritmo a **find-or-create** para permitir evolución sin duplicados.

Roles materializados:

ADMIN

USER

GESTOR

Usuarios:

admin -> ADMIN + USER

ana -> USER

gestor -> GESTOR + USER

gestor es una continuidad deliberada del reto ya completado en 6.2.

Observable. Reiniciar no crea duplicados y tampoco hace desaparecer el reto anterior.

Error que evita la mejora. Una base que ya contenga admin y ana no impediría añadir más adelante GESTOR.

Criterio de cierre

Al cerrar el punto, la autenticación debe depender de la base de datos; la configuración en memoria ya no debe existir.

Paso 6.3.8 - Arrancar y verificar

Objetivo

Completar **arrancar y verificar** y comprobar su efecto antes de continuar. Este punto sustituye los usuarios temporales por entidades JPA y un UserDetailsService propio. El registro debe crear cuentas seguras sin exponer ni aceptar privilegios arbitrarios.

Procedimiento

Cuando el árbol M5 esté completamente materializado:

```
./mvnw spring-boot:run
```

En H2:

```
SELECT * FROM usuarios;  
SELECT * FROM roles;  
SELECT * FROM usuarios_roles;
```

Observable.

- usuarios contiene identidades persistentes;
- roles contiene nombres de rol;
- usuarios_roles contiene pares usuario_id/rol_id.

Pregunta. ¿Por qué la tabla de unión no necesita guardar username ni nombre del rol?

Criterio de cierre

Al cerrar el punto, la autenticación debe depender de la base de datos; la configuración en memoria ya no debe existir.

Paso 6.3.9 - Probar la autenticación

Objetivo

Completar **probar la autenticación** y comprobar su efecto antes de continuar. Este punto sustituye los usuarios temporales por entidades JPA y un UserDetailsService propio. El registro debe crear cuentas seguras sin exponer ni aceptar privilegios arbitrarios.

Procedimiento

```
curl -i -u ana:ana123 \  
http://localhost:8080/api/v1/perfil
```

```
curl -i -u admin:admin123 \  
http://localhost:8080/api/v1/perfil
```

```
curl -i -u gestor:gestor123 \  
http://localhost:8080/api/v1/gestor/documentos
```

Observables.

- ana → ROLE_USER;
- admin → ROLE_ADMIN, ROLE_USER;
- gestor → supera la ruta GESTOR.

Evidencia conceptual. Las mismas credenciales de 6.2 siguen funcionando, pero ya no proceden de InMemoryUserDetailsManager.

Criterio de cierre

Al cerrar el punto, la autenticación debe depender de la base de datos; la configuración en memoria ya no debe existir.

Paso 6.3.10 - Crear los DTOs de registro

Objetivo

Completar **crear los dtos de registro** y comprobar su efecto antes de continuar. Este punto sustituye los usuarios temporales por entidades JPA y un UserDetailsService propio. El registro debe crear cuentas seguras sin exponer ni aceptar privilegios arbitrarios.

Procedimiento

RegistroRequestDTO:

```
@NotBlank
@Size(min = 3, max = 50)
private String username;
```

```
@NotBlank
@Size(min = 8)
private String password;
```

```
@NotBlank
@email
private String email;
```

UsuarioResponseDTO contiene:

```
id
username
email
roles
```

y **no contiene password**.

Pregunta. ¿Por qué no devolvemos siquiera el hash de la contraseña?

Criterio de cierre

Al cerrar el punto, la autenticación debe depender de la base de datos; la configuración en memoria ya no debe existir.

Paso 6.3.11 - Crear AuthService y AuthController

Objetivo

Completar **crear authservice y authcontroller** y comprobar su efecto antes de continuar. Este punto sustituye los usuarios temporales por entidades JPA y un UserDetailsService propio. El registro debe crear cuentas seguras sin exponer ni aceptar privilegios arbitrarios.

Procedimiento

AuthService.registrar(...):

1. rechaza username repetido;
2. rechaza email repetido;

3. exige que exista USER;
4. codifica contraseña;
5. asigna USER;
6. guarda;
7. proyecta a DTO.

El controlador expone:

POST /api/v1/auth/registro

y responde 201 Created.

Continuidad con M5. Se reutilizan las excepciones específicas ya existentes:

- RecursoDuplicadoException;
- RecursoNoEncontradoException;
- OperacionNoPermitidaException para el reto de contraseña.

No se vuelve a introducir la clase genérica NegocioException, que M5 conserva sólo como compatibilidad y marca @Deprecated.

Criterio de cierre

Al cerrar el punto, la autenticación debe depender de la base de datos; la configuración en memoria ya no debe existir.

Paso 6.3.12 - Configurar el endpoint público y probar

Objetivo

Completar **configurar el endpoint público y probar** y comprobar su efecto antes de continuar. Este punto sustituye los usuarios temporales por entidades JPA y un UserDetailsService propio. El registro debe crear cuentas seguras sin exponer ni aceptar privilegios arbitrarios.

Procedimiento

En SecurityConfig usamos un matcher mínimo:

```
.requestMatchers(HttpMethod.POST, "/api/v1/auth/registro").permitAll()
```

No abrimos todavía todo /api/v1/auth/**.

Prueba:

```
curl -i -X POST \
  http://localhost:8080/api/v1/auth/registro \
  -H "Content-Type: application/json" \
  -d '{
    "username": "pedro",
    "password": "pedro1234",
    "email": "pedro@educacion.gob.es"
  }'
```

Observable: 201 Created, sin contraseña en la respuesta.

Después:

```
curl -i -u pedro:pedro1234 \
  http://localhost:8080/api/v1/perfil
```

Debe mostrar ROLE_USER.

Repite el registro con el mismo username o email.

Observable: conflicto de negocio manejado por el sistema de errores heredado de M5.

Criterio de cierre

Al cerrar el punto, la autenticación debe depender de la base de datos; la configuración en memoria ya no debe existir.

Paso 6.3.13 - Errores comunes del ejercicio

Objetivo

Completar **errores comunes del ejercicio** y comprobar su efecto antes de continuar. Este punto sustituye los usuarios temporales por entidades JPA y un UserDetailsService propio. El registro debe crear cuentas seguras sin exponer ni aceptar privilegios arbitrarios.

Procedimiento

Error	Causa	Corrección
más de un UserDetailsService LazyInitializationException	no se cerró 6.2	eliminar InMemoryUserConfig
duplicado al registrar	roles fuera del contexto de persistencia username/email existentes	EAGER o carga explícita dentro de transacción validar en servicio + constraint DB
contraseña visible en BD hasRole siempre falla	faltó encode authorities sin ROLE_	codificar antes de persistir transformar ADMIN → ROLE_ADMIN
registro devuelve 401	matcher no público	permitir sólo POST /api/v1/auth/registro
ciclo de beans	configuración acoplada	constructor injection y responsabilidades separadas
no aparece tabla intermedia roles no guardados usuario inexistente produce error técnico	mapping incompleto se modificó el lado incorrecto excepción incorrecta	revisar @JoinTable añadir al Set de Usuario lanzar UsernameNotFoundException

Comprobación adicional. Registrar no puede permitir que el cliente escoja ADMIN.

Criterio de cierre

Al cerrar el punto, la autenticación debe depender de la base de datos; la configuración en memoria ya no debe existir.

Paso 6.3.14 - Reto resuelto: cambiar la contraseña

Objetivo

Completar **reto resuelto: cambiar la contraseña** y comprobar su efecto antes de continuar. Este punto sustituye los usuarios temporales por entidades JPA y un UserDetailsService propio. El registro debe crear cuentas seguras sin exponer ni aceptar privilegios arbitrarios.

Procedimiento

El reto pide:

PUT /api/v1/perfil/password

y esa será la ruta canónica de nuestra solución.

DTO

```
public class CambioPasswordRequestDTO {  
  
    @NotBlank  
    private String passwordActual;  
  
    @NotBlank  
    @Size(min = 8)  
    private String passwordNueva;  
}
```

Servicio

```
@Transactional  
public void cambiarPassword(  
    String username,  
    CambioPasswordRequestDTO request) {  
  
    Usuario usuario = usuarioRepository.findByUsername(username)  
        .orElseThrow(() ->  
            new RecursoNoEncontradoException("Usuario", username));  
  
    if (!passwordEncoder.matches(  
        request.getPasswordActual(),  
        usuario.getPassword())) {  
        throw new OperacionNoPermitidaException(  
            "La contraseña actual no es correcta");  
    }  
  
    usuario.setPassword(  
        passwordEncoder.encode(request.getPasswordNueva()));  
}
```

No es necesario llamar a save para una entidad gestionada dentro de la transacción, aunque hacerlo tampoco sería incorrecto.

Controlador

El endpoint se añade al PerfilController:

```
@PutMapping("/password")  
public ResponseEntity<Void> cambiarPassword(  
    Authentication authentication,  
    @Valid @RequestBody CambioPasswordRequestDTO request) {  
    authService.cambiarPassword(authentication.getName(), request);  
    return ResponseEntity.noContent().build();  
}
```

Prueba

```
curl -i -X PUT -u pedro:pedro1234 \
  http://localhost:8080/api/v1/perfil/password \
  -H "Content-Type: application/json" \
  -d '{
    "passwordActual": "pedro1234",
    "passwordNueva": "pedro5678"
  }'
```

Observable: 204 No Content.

Después:

```
curl -i -u pedro:pedro5678 \
  http://localhost:8080/api/v1/perfil
```

La nueva contraseña funciona.

La antigua deja de funcionar:

```
curl -i -u pedro:pedro1234 \
  http://localhost:8080/api/v1/perfil
```

Observable: 401.

Decisión técnica

El snippet inicial coloca por error práctico el método bajo `/api/v1/auth/password` mientras el mismo punto abre `/api/v1/auth/**`. Eso podría convertir el cambio de contraseña en endpoint público. Nuestra solución conserva el **enunciado** del reto (`/api/v1/perfil/password`) y garantiza que permanezca autenticado.

Resultado esperado global 6.3

Al cerrar el punto tenemos:

- entidades Usuario y Rol;
- relación @ManyToMany;
- repositorios;
- UsuarioDetailsService persistente;
- inicialización idempotente;
- cierre de usuarios en memoria;
- autenticación con usuarios de base de datos;
- registro público limitado a una ruta;
- cambio de contraseña autenticado;
- continuidad del usuario/rol GESTOR;
- HTTP Basic todavía temporal, pendiente de cierre en 6.7.

6.4 — Autorización por roles

Criterio de cierre

Al cerrar el punto, la autenticación debe depender de la base de datos; la configuración en memoria ya no debe existir.

Práctica 6.4 - Autorización por URL y por método

La meta es separar política HTTP y reglas de negocio. Probaremos 401/403 y combinaremos matchers con `@PreAuthorize` sin confiar en datos de identidad enviados por el cliente.

Paso 6.4.1 - Repasar el estado actual

Objetivo

Completar **repasar el estado actual** y comprobar su efecto antes de continuar. La meta es separar política HTTP y reglas de negocio. Probaremos 401/403 y combinaremos matchers con `@PreAuthorize` sin confiar en datos de identidad enviados por el cliente.

Procedimiento

Venimos de 6.3 con:

- usuarios persistentes;
- `UsuarioDetailsService`;
- roles USER, ADMIN, GESTOR;
- registro público específico;
- `/api/v1/admin/**` protegido por ADMIN;
- HTTP Basic todavía activo;
- CORS, OpenAPI y H2 siguen operativos.

Comprobación estructural:

```
grep -R "InMemoryUserDetailsService" src/main/java || true
```

No debe aparecer.

Pregunta. ¿La regla `/api/v1/admin/**` protege una operación concreta o toda una zona del API?

Criterio de cierre

Al cerrar el punto, un usuario autenticado sin rol debe obtener 403 y un usuario con el rol exigido debe alcanzar la operación.

Paso 6.4.2 - Activar la seguridad por método

Objetivo

Completar **activar la seguridad por método** y comprobar su efecto antes de continuar. La meta es separar política HTTP y reglas de negocio. Probaremos 401/403 y combinaremos matchers con `@PreAuthorize` sin confiar en datos de identidad enviados por el cliente.

Procedimiento

Evoluciona `SecurityConfig`:

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {
    // ...
}
```

Observable. A partir de ahora Spring evalúa @PreAuthorize en beans interceptados.

Error crítico. Añadir @PreAuthorize sin @EnableMethodSecurity: la anotación queda sin el efecto esperado.

Criterio de cierre

Al cerrar el punto, un usuario autenticado sin rol debe obtener 403 y un usuario con el rol exigido debe alcanzar la operación.

Paso 6.4.3 - Crear un UsuarioController de administración

Objetivo

Completar **crear un usuariocontroller de administración** y comprobar su efecto antes de continuar. La meta es separar política HTTP y reglas de negocio. Probaremos 401/403 y combinaremos matchers con @PreAuthorize sin confiar en datos de identidad enviados por el cliente.

Procedimiento

Creamos:

src/main/java/es/mecd/demo/miproyecto/auth/UsuarioController.java

```
@RestController
@RequestMapping("/api/v1/admin/usuarios")
public class UsuarioController {

    private final UsuarioService usuarioService;

    public UsuarioController(UsuarioService usuarioService) {
        this.usuarioService = usuarioService;
    }

    @GetMapping
    @PreAuthorize("hasRole('ADMIN')")
    public List<UsuarioResponseDTO> listar() {
        return usuarioService.listarTodos();
    }

    @GetMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public UsuarioResponseDTO consultar(@PathVariable Long id) {
        return usuarioService.consultar(id);
    }
}
```

Doble barrera.

1. /api/v1/admin/** exige ADMIN en SecurityFilterChain.
2. Los métodos vuelven a exigir ADMIN.

Pregunta. Si la URL ya exige ADMIN, ¿qué protege la anotación ante un futuro cambio de ruta?

Criterio de cierre

Al cerrar el punto, un usuario autenticado sin rol debe obtener 403 y un usuario con el rol exigido debe alcanzar la operación.

Paso 6.4.4 - Crear el UsuarioService

Objetivo

Completar **crear el usuarioservice** y comprobar su efecto antes de continuar. La meta es separar política HTTP y reglas de negocio. Probaremos 401/403 y combinaremos matchers con @PreAuthorize sin confiar en datos de identidad enviados por el cliente.

Procedimiento

El servicio necesita UsuarioRepository para leer usuarios y RolRepository para el reto de cambio de roles.

```
@Service
public class UsuarioService {
    // ...

    @Transactional(readOnly = true)
    public List<UsuarioResponseDTO> listarTodos() {
        return usuarioRepository.findAll().stream()
            .map(this::toDTO)
            .toList();
    }

    @Transactional(readOnly = true)
    public UsuarioResponseDTO consultar(Long id) {
        return usuarioRepository.findById(id)
            .map(this::toDTO)
            .orElseThrow(() ->
                new RecursoNoEncontradoException("Usuario", id));
    }
}
```

El mapeo ordena los nombres de rol para producir respuestas deterministas.

Pregunta. ¿Podría existir @PreAuthorize también en el servicio? Sí. La capa de método no está limitada a controladores; la decisión depende de dónde queramos fijar el límite de seguridad.

Criterio de cierre

Al cerrar el punto, un usuario autenticado sin rol debe obtener 403 y un usuario con el rol exigido debe alcanzar la operación.

Paso 6.4.5 - Añadir reglas de URL y de método combinadas

Objetivo

Completar **añadir reglas de url y de método combinadas** y comprobar su efecto antes de continuar. La meta es separar política HTTP y reglas de negocio. Probaremos 401/403 y combinaremos matchers con @PreAuthorize sin confiar en datos de identidad enviados por el cliente.

Procedimiento

La configuración acumulativa conserva:

```
.requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
.anyRequest().authenticated()
```

y el controlador añade:

```
@PreAuthorize("hasRole('ADMIN')")
```

Observable. Un USER puede ser rechazado por la cadena antes de que el controlador llegue siquiera a evaluarse.

Pregunta. ¿Por qué esto no significa que @PreAuthorize sea inútil?

Criterio de cierre

Al cerrar el punto, un usuario autenticado sin rol debe obtener 403 y un usuario con el rol exigido debe alcanzar la operación.

Paso 6.4.6 - Probar la autorización con distintos usuarios

Objetivo

Completar **probar la autorización con distintos usuarios** y comprobar su efecto antes de continuar. La meta es separar política HTTP y reglas de negocio. Probaremos 401/403 y combinaremos matchers con @PreAuthorize sin confiar en datos de identidad enviados por el cliente.

Procedimiento

```
# USER autenticado
curl -i -u ana:ana123 \
  http://localhost:8080/api/v1/admin/usuarios

# ADMIN
curl -i -u admin:admin123 \
  http://localhost:8080/api/v1/admin/usuarios

# Anónimo
curl -i \
  http://localhost:8080/api/v1/admin/usuarios
```

Esperado:

```
ana      -> 403
admin    -> 200
anónimo  -> 401
```

Pregunta. ¿Por qué Ana recibe 403 y no 401?

Criterio de cierre

Al cerrar el punto, un usuario autenticado sin rol debe obtener 403 y un usuario con el rol exigido debe alcanzar la operación.

Paso 6.4.7 - Crear un endpoint para consultar el propio perfil

Objetivo

Completar **crear un endpoint para consultar el propio perfil** y comprobar su efecto antes de continuar. La meta es separar política HTTP y reglas de negocio. Probaremos 401/403 y combinaremos matchers con @PreAuthorize sin confiar en datos de identidad enviados por el cliente.

Procedimiento

El perfil ya existe desde 6.2/6.3. Ajústalo para trabajar con el principal autenticado:

```
@GetMapping
public Map<String, Object> perfil(
    @AuthenticationPrincipal UserDetails user) {
    return Map.of(
        "usuario", user.getUsername(),
        "authorities", user.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority)
            .sorted()
            .toList());
}
```

El cambio de contraseña conserva Authentication, porque en ese método sólo necesitamos getName() y su contrato ya está probado.

Observable. @AuthenticationPrincipal entrega el UserDetails que Spring colocó en el contexto.

Pregunta. ¿Cuándo preferirías recibir Authentication completo?

Criterio de cierre

Al cerrar el punto, un usuario autenticado sin rol debe obtener 403 y un usuario con el rol exigido debe alcanzar la operación.

Paso 6.4.8 - Crear UsuarioPrincipal para 6.7

Objetivo

Completar **crear usuarioprincipal para 6.7** y comprobar su efecto antes de continuar. La meta es separar política HTTP y reglas de negocio. Probaremos 401/403 y combinaremos matchers con @PreAuthorize sin confiar en datos de identidad enviados por el cliente.

Procedimiento

Creemos ahora, sin integrarlo todavía:

```
public class UsuarioPrincipal implements UserDetails {
    private final Long id;
    private final String username;
    private final String email;
    private final Collection<? extends GrantedAuthority> authorities;

    // constructor + getters + contrato UserDetails
}
```

getPassword() devuelve null: este principal está diseñado para representar una identidad ya validada desde token en 6.7, no para almacenar credenciales.

Preparación para el filtro JWT.

- La clase existe.
- UsuarioDetailsService sigue devolviendo User estándar.
- La integración se cierra en **6.7**.

Pregunta. ¿Por qué no sustituimos ya el principal activo si el ejercicio introduce primero la transición?

Criterio de cierre

Al cerrar el punto, un usuario autenticado sin rol debe obtener 403 y un usuario con el rol exigido debe alcanzar la operación.

Paso 6.4.9 - Probar la expresión de autorización

Objetivo

Completar **probar la expresión de autorización** y comprobar su efecto antes de continuar. La meta es separar política HTTP y reglas de negocio. Probaremos 401/403 y combinaremos matchers con `@PreAuthorize` sin confiar en datos de identidad enviados por el cliente.

Procedimiento

El ejercicio propone la expresión futura:

```
#id == authentication.principal.id
```

pero también advierte que **todavía no puede funcionar** con el principal actual, porque User de Spring Security no tiene `getId()`.

Por tanto este paso se ejecuta como **prueba negativa controlada**, no como código final roto.

Si se añade temporalmente una expresión que accede a:

```
authentication.principal.id
```

la evaluación puede lanzar `SpelEvaluationException`.

Restauración obligatoria. Elimina la expresión temporal antes de continuar. No queda ningún endpoint final de 6.4 dependiendo de `principal.id`.

El caso funcional se resolverá en 6.7, donde el principal será `UsuarioPrincipal`.

Criterio de cierre

Al cerrar el punto, un usuario autenticado sin rol debe obtener 403 y un usuario con el rol exigido debe alcanzar la operación.

Paso 6.4.10 - Escribir tests de autorización

Objetivo

Completar **escribir tests de autorización** y comprobar su efecto antes de continuar. La meta es separar política HTTP y reglas de negocio. Probaremos 401/403 y combinaremos matchers con `@PreAuthorize` sin confiar en datos de identidad enviados por el cliente.

Procedimiento

En Spring Boot 3.5 utiliza `@MockitoBean` para declarar el mock dentro del contexto del test.

```
@WebMvcTest(UsuarioController.class)
```

```
@Import(SecurityConfig.class)
```

```
class UsuarioControllerTest {
```

```
    @Autowired
```

```
    private MockMvc mockMvc;
```

```
    @MockitoBean
```

```
    private UsuarioService usuarioService;
```

```

@Test
void listar_debeDevolver403_cuandoNoEsAdmin()
    throws Exception {
    mockMvc.perform(get("/api/v1/admin/usuarios")
        .with(user("ana").roles("USER")))
        .andExpect(status().isForbidden());
}

@Test
void listar_debeDevolver200_cuandoEsAdmin()
    throws Exception {
    mockMvc.perform(get("/api/v1/admin/usuarios")
        .with(user("admin").roles("ADMIN")))
        .andExpect(status().isOk());
}

@Test
void listar_debeDevolver401_cuandoNoAutenticado()
    throws Exception {
    mockMvc.perform(get("/api/v1/admin/usuarios"))
        .andExpect(status().isUnauthorized());
}
}

```

Objetivo. Cubrir las tres ramas semánticamente distintas: 401, 403 y 200.

Aislamiento real del slice MVC. `@WebMvcTest` no levanta toda la aplicación, pero sí puede descubrir componentes web como `JwtAuthenticationFilter`. Además, `@AutoConfigureMockMvc(addFilters = false)` evita aplicar los filtros a las peticiones de `MockMvc`, pero **no impide que Spring cree el bean del filtro** durante el arranque del contexto. Como el filtro recibe `JwtService` y `TokenRevocationService` por constructor, el slice debe proporcionar esos colaboradores con `@MockitoBean` cuando no carga la configuración completa de M6.

```

@MockitoBean
private JwtService jwtService;

```

```

@MockitoBean
private TokenRevocationService tokenRevocationService;

```

Así este test sigue verificando únicamente 401/403/200 del controlador y de su `TestSecurityConfig`, mientras los tests JWT dedicados prueban el mecanismo real. Esta separación evita convertir un slice rápido en un `@SpringBootTest` y, sobre todo, evita un falso negativo de arranque por una dependencia que no forma parte del objetivo del test.

Criterio de cierre

Al cerrar el punto, un usuario autenticado sin rol debe obtener 403 y un usuario con el rol exigido debe alcanzar la operación.

Paso 6.4.11 - Errores comunes del ejercicio

Objetivo

Completar **errores comunes del ejercicio** y comprobar su efecto antes de continuar. La meta es separar política HTTP y reglas de negocio. Probaremos 401/403 y combinaremos matchers con `@PreAuthorize` sin confiar en datos de identidad enviados por el cliente.

Procedimiento

Error	Causa	Solución
@PreAuthorize no se aplica hasRole("ROLE_ADMIN") falla hasAuthority("ADMIN") falla SpEL no resuelve id 403 con rol esperado	falta @EnableMethodSecurity hasRole gestiona el prefijo authority real es ROLE_ADMIN principal actual no expone id transformación ROLE_ inconsistente	activarlo usar hasRole("ADMIN") usar literal correcto esperar a UsuarioPrincipal revisar UsuarioDetailsService
endpoint anónimo da 200	falta regla general	mantener anyRe- quest().authenticated()
anotación en privado	proxy no intercepta como se espera	proteger método público
llamada interna elude seguridad	self-invocation	mover frontera a otro bean
test usa API obsoleta test 401/403 confundido	@MockBean antiguo identidad/rol mal preparados	usar @MockitoBean separar escenario anónimo y autenticado

Criterio de cierre

Al cerrar el punto, un usuario autenticado sin rol debe obtener 403 y un usuario con el rol exigido debe alcanzar la operación.

Paso 6.4.12 - Reto resuelto: endpoint de cambio de rol

Objetivo

Completar **reto resuelto: endpoint de cambio de rol** y comprobar su efecto antes de continuar. La meta es separar política HTTP y reglas de negocio. Probaremos 401/403 y combinaremos matchers con @PreAuthorize sin confiar en datos de identidad enviados por el cliente.

Procedimiento

El reto exige:

PUT /api/v1/admin/usuarios/{id}/roles

DTO

```
public class CambioRolesRequestDTO {
    @NotEmpty(message = "Debe indicar al menos un rol")
    private List<String> roles;

    // getter/setter
}
```

Servicio

```
@Transactional
public UsuarioResponseDTO cambiarRoles(
    Long id,
    List<String> nombresRoles) {

    Usuario usuario = usuarioRepository.findById(id)
```

```

        .orElseThrow(() ->
            new RecursoNoEncontradoException(
                "Usuario", id));

    Set<Rol> roles = new HashSet<>();
    for (String nombre : nombresRoles) {
        Rol rol = rolRepository.findByNombre(nombre)
            .orElseThrow(() ->
                new RecursoNoEncontradoException(
                    "Rol", nombre));
        roles.add(rol);
    }

    usuario.setRoles(roles);
    return toDTO(usuario);
}

```

Dentro de la transacción no necesitamos un save redundante para una entidad gestionada; dejamos que el dirty checking persista el cambio.

Controlador

```

@PutMapping("/{id}/roles")
@PreAuthorize("hasRole('ADMIN')")
public UsuarioResponseDTO cambiarRoles(
    @PathVariable Long id,
    @Valid @RequestBody CambioRolesRequestDTO request) {
    return usuarioService.cambiarRoles(
        id,
        request.getRoles());
}

```

Prueba

```

curl -i -X PUT -u admin:admin123 \
  http://localhost:8080/api/v1/admin/usuarios/2/roles \
  -H "Content-Type: application/json" \
  -d '{"roles":["USER","GESTOR"]}'

```

Observable: el usuario queda con el conjunto exacto de roles solicitado.

Prueba negativa:

```

curl -i -X PUT -u ana:ana123 \
  http://localhost:8080/api/v1/admin/usuarios/2/roles \
  -H "Content-Type: application/json" \
  -d '{"roles":["ADMIN"]}'

```

Observable: 403.

Resultado esperado global 6.4

- @EnableMethodSecurity activo;
- administración por URL + método;
- UsuarioService;
- perfil con @AuthenticationPrincipal;
- UsuarioPrincipal creado pero todavía no activo;
- prueba negativa de SpEL cerrada/restaurada;

- tests 401/403/200 con @MockitoBean;
- cambio de roles sólo para ADMIN.

6.5 — Introducción a JWT

El código de este punto es **didáctico**. No se convierte en el mecanismo de autenticación productivo. Su objetivo es entender Base64Url y HMAC antes de usar JWT en 6.6.

Criterio de cierre

Al cerrar el punto, un usuario autenticado sin rol debe obtener 403 y un usuario con el rol exigido debe alcanzar la operación.

Práctica 6.5 - JWT: estructura, firma y ciclo de vida

Antes de usar una biblioteca conviene entender qué se firma, qué puede leerse sin clave y qué errores invalidan un token. Los experimentos manuales son deliberadamente temporales.

Paso 6.5.1 - Entender la estructura con un ejemplo

Objetivo

Completar **entender la estructura con un ejemplo** y comprobar su efecto antes de continuar. Antes de usar una biblioteca conviene entender qué se firma, qué puede leerse sin clave y qué errores invalidan un token. Los experimentos manuales son deliberadamente temporales.

Procedimiento

El ejercicio propone inspeccionar este JWT didáctico:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.  
eyJzdWIiOiJhbmEiLCJyb2xlcYI6WyJST0xFX1VTRVIiXSwiaWF0IjoxeNTE2MjM5MDIyfQ.  
SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```

Sin los saltos de línea forma una sola cadena.

Header:

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

Payload:

```
{  
  "sub": "ana",  
  "roles": ["ROLE_USER"],  
  "iat": 1516239022  
}
```

Observable. Header y payload son legibles sin conocer la clave.

El ejemplo usa `jwt.io` para visualizarlo. Sólo se deben pegar allí **tokens didácticos**, nunca access/refresh tokens reales de un sistema.

Pregunta. Si el payload se puede leer, ¿qué aporta entonces la firma?

Criterio de cierre

Al cerrar el punto, debes distinguir decodificar de verificar y demostrar que modificar el payload rompe la firma.

Paso 6.5.2 - Crear una clase para generar un JWT manualmente

Objetivo

Completar **crear una clase para generar un jwt manualmente** y comprobar su efecto antes de continuar. Antes de usar una biblioteca conviene entender qué se firma, qué puede leerse sin clave y qué errores invalidan un token. Los experimentos manuales son deliberadamente temporales.

Procedimiento

Creemos en código de pruebas:

```
src/test/java/es/mecd/demo/miproyecto/jwt/JwtManual.java
```

La clase usa exclusivamente Java estándar:

```
Base64.getUrlEncoder().withoutPadding()  
Mac.getInstance("HmacSHA256")  
SecretKeySpec
```

Flujo:

```
JSON header  
  ↓ Base64Url  
header64
```

```
JSON payload  
  ↓ Base64Url  
payload64
```

```
header64.payload64  
  ↓ HMAC-SHA256(secret)  
signature64
```

```
header64.payload64.signature64
```

La constante incluida en la clase está marcada DIDACTIC_SECRET: es una clave de ejemplo y **no** un secreto de producción.

Experimento JWT manual. Firma JWT manual sólo para aprendizaje. La implementación operativa se reemplaza por JJWT en 6.6.

Criterio de cierre

Al cerrar el punto, debes distinguir decodificar de verificar y demostrar que modificar el payload rompe la firma.

Paso 6.5.3 - Decodificar el token generado

Objetivo

Completar **decodificar el token generado** y comprobar su efecto antes de continuar. Antes de usar una biblioteca conviene entender qué se firma, qué puede leerse sin clave y qué errores invalidan un token. Los experimentos manuales son deliberadamente temporales.

Procedimiento

La utilidad incorpora:

```
public static String decodificarPayload(String jwt) {
    String[] partes = jwt.split("\\.", -1);
    if (partes.length != 3) {
        throw new IllegalArgumentException("Token mal formado");
    }

    byte[] decoded = Base64.getUrlDecoder()
        .decode(partes[1]);
    return new String(decoded, StandardCharsets.UTF_8);
}
```

Ejecuta la clase:

```
./mvnw -Dtest=JwtManualTest test
```

o su main desde el IDE.

Observable. El JSON del payload aparece en texto plano.

Pregunta. ¿Ha sido necesaria la clave para leerlo?

Criterio de cierre

Al cerrar el punto, debes distinguir decodificar de verificar y demostrar que modificar el payload rompe la firma.

Paso 6.5.4 - Modificar el payload sin actualizar la firma

Objetivo

Completar **modificar el payload sin actualizar la firma** y comprobar su efecto antes de continuar. Antes de usar una biblioteca conviene entender qué se firma, qué puede leerse sin clave y qué errores invalidan un token. Los experimentos manuales son deliberadamente temporales.

Procedimiento

Toma un token válido y sustituye sólo su segunda parte por otro payload Base64Url, conservando la firma antigua.

Ejemplo conceptual:

```
header.payload_original.firma_original
      ↓ cambiar
header.payload_modificado.firma_original
```

Observable. La decodificación sigue funcionando, pero la verificación criptográfica falla.

Esto demuestra por qué:

payload legible ≠ payload confiable

Restauración. El token manipulado sólo se usa como dato de prueba y no se reutiliza en pasos posteriores como token válido.

Criterio de cierre

Al cerrar el punto, debes distinguir decodificar de verificar y demostrar que modificar el payload rompe la firma.

Paso 6.5.5 - Verificar el token con Java

Objetivo

Completar **verificar el token con java** y comprobar su efecto antes de continuar. Antes de usar una biblioteca conviene entender qué se firma, qué puede leerse sin clave y qué errores invalidan un token. Los experimentos manuales son deliberadamente temporales.

Procedimiento

`JwtManual.verificar(jwt, secret):`

1. exige exactamente tres partes;
2. vuelve a firmar `header.payload`;
3. decodifica firma esperada y recibida;
4. compara los bytes con `MessageDigest.isEqual`.

Ejemplo:

```
boolean valido = JwtManual.verificar(
    jwt,
    JwtManual.DIDACTIC_SECRET);
```

Observable. Token intacto → true; token manipulado → false.

Importante. Esta verificación manual valida la **firma**, no interpreta todavía exp. La expiración real será validada por la biblioteca en 6.6.

Criterio de cierre

Al cerrar el punto, debes distinguir decodificar de verificar y demostrar que modificar el payload rompe la firma.

Paso 6.5.6 - Probar con distintos payloads

Objetivo

Completar **probar con distintos payloads** y comprobar su efecto antes de continuar. Antes de usar una biblioteca conviene entender qué se firma, qué puede leerse sin clave y qué errores invalidan un token. Los experimentos manuales son deliberadamente temporales.

Procedimiento

El ejercicio amplía los claims:

```
{
  "sub": "ana",
  "email": "ana@educacion.gob.es",
  "roles": ["ROLE_USER", "ROLE_GESTOR"],
  "iat": 1516239022,
  "exp": 9999999999
}
```

Genera el token y decodifícalo localmente.

Observables.

- sub, iat y exp son claims registrados;
- email y roles son datos de aplicación en este ejemplo;
- añadir claims aumenta el tamaño del token;
- el contenido sigue siendo legible.

Pregunta. ¿Qué dato de esa lista no debería añadirse si no es necesario para tomar decisiones?

Criterio de cierre

Al cerrar el punto, debes distinguir decodificar de verificar y demostrar que modificar el payload rompe la firma.

Paso 6.5.7 - Reflexionar sobre la seguridad

Objetivo

Completar **reflexionar sobre la seguridad** y comprobar su efecto antes de continuar. Antes de usar una biblioteca conviene entender qué se firma, qué puede leerse sin clave y qué errores invalidan un token. Los experimentos manuales son deliberadamente temporales.

Procedimiento

Responde y justifica:

1. ¿Se puede leer el payload sin la clave? **Sí**, porque Base64Url no cifra.
2. ¿Se puede cambiar sin invalidar la firma? **No**, salvo que el atacante pueda crear una firma válida.
3. ¿Qué ocurre si se filtra una clave HS256? Un atacante podría emitir tokens que el servidor aceptaría como firmados por nosotros.
4. ¿Debe viajar una contraseña dentro del payload? **No**.
5. ¿Un token firmado necesita HTTPS? **Sí**: la firma no impide que otro capture y reutilice el token.
6. ¿Debemos registrar tokens completos en logs? **No**.

Observable. El modelo de amenaza queda explícito antes de introducir la librería.

Criterio de cierre

Al cerrar el punto, debes distinguir decodificar de verificar y demostrar que modificar el payload rompe la firma.

Paso 6.5.8 - Comparar con un token real

Objetivo

Completar **comparar con un token real** y comprobar su efecto antes de continuar. Antes de usar una biblioteca conviene entender qué se firma, qué puede leerse sin clave y qué errores invalidan un token. Los experimentos manuales son deliberadamente temporales.

Procedimiento

En 6.6 JWT hará de forma robusta lo que aquí hemos hecho manualmente:

- construir/serializar claims;
- codificar las partes;
- firmar;
- parsear;
- verificar firma;
- verificar expiración;
- convertir errores en excepciones específicas.

La estructura compacta seguirá siendo:

`header.payload.signature`

Cierre conceptual. Comprender la implementación manual sirve para diagnosticar; no justifica reimplementar JWT en código productivo.

Criterio de cierre

Al cerrar el punto, debes distinguir decodificar de verificar y demostrar que modificar el payload rompe la firma.

Paso 6.5.9 - Errores comunes del ejercicio

Objetivo

Completar **errores comunes del ejercicio** y comprobar su efecto antes de continuar. Antes de usar una biblioteca conviene entender qué se firma, qué puede leerse sin clave y qué errores invalidan un token. Los experimentos manuales son deliberadamente temporales.

Procedimiento

Error	Causa	Corrección
<code>IllegalArgumentException</code> <code>Base64</code> firma no coincide	alfabeto/formato incorrecto	usar <code>getUrlEncoder/getUrlDecoder</code>
aparece = al final no hay tres partes	clave o contenido distintos	verificar <code>header.payload</code> y clave <code>withoutPadding()</code>
<code>NoSuchAlgorithmException</code> caracteres extraños "decodifica, luego es válido"	encoder estándar/padding token mal formado	validar estructura antes de acceder <code>HmacSHA256</code>
token real enviado a web externa	nombre de algoritmo incorrecto charset implícito confundir lectura con verificación exposición de credencial	usar UTF-8 verificar firma usar tokens de ejemplo o decoder local

Criterio de cierre

Al cerrar el punto, debes distinguir decodificar de verificar y demostrar que modificar el payload rompe la firma.

Paso 6.5.10 - Experimentar con la expiración

Objetivo

Completar **experimentar con la expiración** y comprobar su efecto antes de continuar. Antes de usar una biblioteca conviene entender qué se firma, qué puede leerse sin clave y qué errores invalidan un token. Los experimentos manuales son deliberadamente temporales.

Procedimiento

Genera un payload:

```
{
  "sub": "ana",
```

```
"exp": 1
}
```

Al decodificarlo se ve que exp representa un instante muy antiguo.

Punto crítico. Nuestra función manual verificar sólo comprueba HMAC y, por diseño pedagógico, podría devolver true aunque exp esté vencido.

Esto no contradice JWT: demuestra que el consumidor debe validar **firma + claims temporales**. JWT se encargará de esa política en 6.6.

Pregunta. ¿Qué peligro tendría verificar únicamente la firma?

Criterio de cierre

Al cerrar el punto, debes distinguir decodificar de verificar y demostrar que modificar el payload rompe la firma.

Paso 6.5.11 - Verificar el token con una clave incorrecta

Objetivo

Completar **verificar el token con una clave incorrecta** y comprobar su efecto antes de continuar. Antes de usar una biblioteca conviene entender qué se firma, qué puede leerse sin clave y qué errores invalidan un token. Los experimentos manuales son deliberadamente temporales.

Procedimiento

```
boolean valido = JwtManual.verificar(
    jwt,
    "otraClaveQueNoEsLaOriginal");
```

Observable: false.

Con HS256, la capacidad de generar una firma válida depende del secreto compartido.

Pregunta. ¿Por qué una fuga de esa clave es más grave que la exposición del payload?

Criterio de cierre

Al cerrar el punto, debes distinguir decodificar de verificar y demostrar que modificar el payload rompe la firma.

Paso 6.5.12 - Reto resuelto: decodificar un token sin verificar la firma

Objetivo

Completar **reto resuelto: decodificar un token sin verificar la firma** y comprobar su efecto antes de continuar. Antes de usar una biblioteca conviene entender qué se firma, qué puede leerse sin clave y qué errores invalidan un token. Los experimentos manuales son deliberadamente temporales.

Procedimiento

El método queda implementado en JwtManual:

```
public static String decodificarPayload(String jwt) {
    String[] partes = jwt.split("\\.", -1);
    if (partes.length != 3) {
        throw new IllegalArgumentException("Token mal formado");
    }
}
```

```
}  
byte[] decoded = Base64.getUrlDecoder()  
    .decode(partes[1]);  
return new String(decoded, StandardCharsets.UTF_8);  
}
```

Uso:

```
String payload = JwtManual.decodificarPayload(jwt);  
System.out.println(payload);
```

Regla de seguridad.

DECODIFICAR = inspeccionar

VERIFICAR = decidir si podemos confiar

Nunca se debe autorizar una operación basándose únicamente en un payload decodificado.

Resultado esperado global 6.5

Al finalizar:

- comprendemos header/payload/signature;
- hemos generado JWT manual con Java estándar;
- hemos verificado HMAC-SHA256;
- modificar payload invalida firma;
- clave incorrecta invalida firma;
- Base64Url no cifra;
- exp necesita validación explícita;
- podemos decodificar localmente sin verificar;
- la implementación manual queda clasificada como **didáctica** y será sustituida como mecanismo real por JJWT en 6.6.

6.6 — Generación de tokens JWT

Criterio de cierre

Al cerrar el punto, debes distinguir decodificar de verificar y demostrar que modificar el payload rompe la firma.

Práctica 6.6 - Generación y validación de JWT con JJWT

Pasamos de los experimentos a una implementación reutilizable con JJWT 0.13.0, claves externalizadas, login, refresh y revocación controlada.

Paso 6.6.1 - Añadir las dependencias de JJWT

Objetivo

Completar **añadir las dependencias de jjwt** y comprobar su efecto antes de continuar. Pasamos de los experimentos a una implementación reutilizable con JJWT 0.13.0, claves externalizadas, login, refresh y revocación controlada.

Procedimiento

Archivo: M6/proyecto/pom.xml.

Usa JJWT **0.13.0**: jjwt-api en compile y jjwt-impl/jjwt-jackson en runtime.

```
./mvnw -B -DskipTests compile
```

Observable: Maven resuelve los tres módulos y el código puede importar Jwts, Claims, Decoders y Keys.

Error/corrección: si falta un módulo runtime, la compilación puede pasar pero el parseo/serialización falla al ejecutar; conservar los tres.

Pregunta: ¿por qué la implementación y Jackson no necesitan exponerse al compilador de nuestro código?

Criterio de cierre

Al cerrar el punto, login y refresh deben emitir el contrato de tokens esperado y el perfil de producción no debe contener un secreto de fallback.

Paso 6.6.2 - Configurar las propiedades de JWT

Objetivo

Completar **configurar las propiedades de jwt** y comprobar su efecto antes de continuar. Pasamos de los experimentos a una implementación reutilizable con JJWT 0.13.0, claves externalizadas, login, refresh y revocación controlada.

Procedimiento

M6 añade `jwt.secret`, `jwt.expiration` y `jwt.refresh-expiration` a `dev/test/prod`. Dev y test usan secretos Base64 no productivos; producción exige `${JWT_SECRET}`.

```
jwt.secret=${JWT_SECRET}
jwt.expiration=${JWT_EXPIRATION:9000000}
jwt.refresh-expiration=${JWT_REFRESH_EXPIRATION:604800000}
```

Observable: prod no tiene secreto embebido; si `JWT_SECRET` falta o no es Base64 fuerte, el arranque falla cerrado.

Decisión técnica: si `JwtConfig` decodifica Base64, el fallback de desarrollo debe ser Base64 real. M6 elimina la contradicción “texto cualquiera + `Decoders.BASE64`”.

Criterio de cierre

Al cerrar el punto, login y refresh deben emitir el contrato de tokens esperado y el perfil de producción no debe contener un secreto de fallback.

Paso 6.6.3 - Crear la configuración de la clave

Objetivo

Completar **crear la configuración de la clave** y comprobar su efecto antes de continuar. Pasamos de los experimentos a una implementación reutilizable con JJWT 0.13.0, claves externalizadas, login, refresh y revocación controlada.

Procedimiento

Archivo: `auth/JwtConfig.java`.

```
@Bean
SecretKey jwtSecretKey() {
    return Keys.hmacShaKeyFor(Decoders.BASE64.decode(secret));
}
```

La implementación captura errores de formato y los convierte en un fallo de configuración explícito.

Observable: una clave <256 bits o Base64 inválido impide iniciar un sistema aparentemente seguro con una clave débil.

Criterio de cierre

Al cerrar el punto, login y refresh deben emitir el contrato de tokens esperado y el perfil de producción no debe contener un secreto de fallback.

Paso 6.6.4 - Crear el JwtService

Objetivo

Completar **crear el jwtservice** y comprobar su efecto antes de continuar. Pasamos de los experimentos a una implementación reutilizable con JWT 0.13.0, claves externalizadas, login, refresh y revocación controlada.

Procedimiento

Archivo: auth/JwtService.java.

Implementa:

- generarToken(Usuario);
- generarRefreshToken(Usuario);
- extraerUsername / extraerId / extraerClaims;
- extraerAuthorities;
- esValido;
- esAccessTokenValido;
- esRefreshTokenValido.

El access contiene tipo=access, jti, id/email/roles; refresh contiene tipo=refresh y jti, sin roles.

```
Claims claims = Jwts.parser()
    .verifyWith(clave)
    .build()
    .parseSignedClaims(token)
    .getPayload();
```

Cierre del experimento manual. A partir de aquí JWT es la implementación operativa; JwtManual queda sólo como ejercicio/test.

Error/corrección: un claim numérico puede llegar como cualquier Number; M6 convierte con longValue() y no asume que siempre sea Long.

Criterio de cierre

Al cerrar el punto, login y refresh deben emitir el contrato de tokens esperado y el perfil de producción no debe contener un secreto de fallback.

Paso 6.6.5 - Crear los DTOs de login

Objetivo

Completar **crear los dtos de login** y comprobar su efecto antes de continuar. Pasamos de los experimentos a una implementación reutilizable con JWT 0.13.0, claves externalizadas, login, refresh y revocación controlada.

Procedimiento

Archivos:

- LoginRequestDTO;
- LoginResponseDTO;
- RefreshRequestDTO.

LoginResponseDTO devuelve snake_case con @JsonProperty y expires_in en segundos.

```
{
  "access_token": "...",
  "refresh_token": "...",
  "token_type": "Bearer",
  "expires_in": 3600
}
```

Error/corrección: no devolver expires_in=3600000; convertir ms → s.

Criterio de cierre

Al cerrar el punto, login y refresh deben emitir el contrato de tokens esperado y el perfil de producción no debe contener un secreto de fallback.

Paso 6.6.6 - Ampliar AuthService con login y refresh

Objetivo

Completar **ampliar authservice con login y refresh** y comprobar su efecto antes de continuar. Pasamos de los experimentos a una implementación reutilizable con JWT 0.13.0, claves externalizadas, login, refresh y revocación controlada.

Procedimiento

AuthService.login usa AuthenticationManager; no compara passwords manualmente. Captura AuthenticationException y devuelve una excepción funcional 401 sin enumerar usuarios.

refresh exige:

1. firma/expiración válidas;
2. tipo=refresh;
3. jti no consumido;
4. usuario aún existente;
5. generar un nuevo access y **nuevo refresh**.

TokenRevocationService.consumeRefresh hace que el refresh anterior sea de un solo uso dentro del proceso.

Decisión técnica: jwtService.esValido() por sí solo permitiría usar un access token como refresh. M6 separa tipos y lo rechaza.

Referencia de implementación

```
usuario.setPassword(passwordEncoder.encode(request.getPassword()));
usuario.setEmail(request.getEmail());
usuario.setActivo(true);
usuario.getRoles().add(rolUser);
return toDTO(usuarioRepository.save(usuario));
}
```

```
@Transactional(readOnly = true)
public LoginResponseDTO login(LoginRequestDTO request) {
    try {
        authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                request.getUsername(), request.getPassword());
    } catch (AuthenticationException ex) {
        throw new CredencialesInvalidasException();
    }

    Usuario usuario = usuarioRepository.findByUsername(request.getUsername())
        .orElseThrow(CredencialesInvalidasException::new);
    return crearPar(usuario);
}

@Transactional(readOnly = true)
public LoginResponseDTO refresh(String refreshToken) {
    if (!jwtService.esRefreshTokenValido(refreshToken)) {
        throw new TokenInvalidoException("Refresh token inválido o expirado");
    }

    Claims claims = jwtService.extraerClaims(refreshToken);
    String jti = claims.getId();
    if (jti == null || !tokenRevocationService.consumeRefresh(
        jti, claims.getExpiration().toInstant())) {
        throw new TokenInvalidoException("Refresh token ya utilizado");
    }

    String username = claims.getSubject();
    Usuario usuario = usuarioRepository.findByUsername(username)
        .orElseThrow(() -> new RecursoNoEncontradoException("Usuario", username));
    if (!usuario.isActivo()) {
        throw new TokenInvalidoException("Usuario inactivo");
    }
    return crearPar(usuario);
}

public void logout(String accessToken) {
    if (!jwtService.esAccessTokenValido(accessToken)) {
        throw new TokenInvalidoException("Access token inválido o expirado");
    }

    Claims claims = jwtService.extraerClaims(accessToken);
    tokenRevocationService.revokeAccess(
        claims.getId(), claims.getExpiration().toInstant());
}

@Transactional
public void cambiarPassword(
    String username,
    CambioPasswordRequestDTO request) {
    Usuario usuario = usuarioRepository.findByUsername(username)
        .orElseThrow(() -> new RecursoNoEncontradoException("Usuario", username));

    if (!passwordEncoder.matches(
        request.getPasswordActual(), usuario.getPassword())) {
```

```

        throw new OperacionNoPermitidaException(
            "La contraseña actual no es correcta");
    }

    usuario.setPassword(passwordEncoder.encode(request.getPasswordNueva()));
}

private LoginResponseDTO crearPar(Usuario usuario) {
    return new LoginResponseDTO(
        jwtService.generarToken(usuario),
        jwtService.generarRefreshToken(usuario),
        jwtService.getExpirationMs() / 1000);
}

private UsuarioResponseDTO toDTO(Usuario usuario) {
    UsuarioResponseDTO dto = new UsuarioResponseDTO();
    dto.setIdentificador(usuario.getId() == null ? null : usuario.getId().toString());
    dto.setUsername(usuario.getUsername());
    dto.setEmail(usuario.getEmail());
    dto.setRoles(usuario.getRoles().stream()
        .map(Rol::getNombre)
        .sorted()
        .toList());
    return dto;
}
}

```

Criterio de cierre

Al cerrar el punto, login y refresh deben emitir el contrato de tokens esperado y el perfil de producción no debe contener un secreto de fallback.

Paso 6.6.7 - Ampliar AuthController

Objetivo

Completar **ampliar authcontroller** y comprobar su efecto antes de continuar. Pasamos de los experimentos a una implementación reutilizable con JJWT 0.13.0, claves externalizadas, login, refresh y revocación controlada.

Procedimiento

Endpoints:

```

POST /api/v1/auth/registro
POST /api/v1/auth/login
POST /api/v1/auth/refresh
POST /api/v1/auth/logout

```

Login y refresh reciben DTO validado; logout extrae el Bearer y delega revocación.

Criterio de cierre

Al cerrar el punto, login y refresh deben emitir el contrato de tokens esperado y el perfil de producción no debe contener un secreto de fallback.

Paso 6.6.8 - Configurar la seguridad para permitir login

Objetivo

Completar **configurar la seguridad para permitir login** y comprobar su efecto antes de continuar. Pasamos de los experimentos a una implementación reutilizable con JWT 0.13.0, claves externalizadas, login, refresh y revocación controlada.

Procedimiento

M6 **no** abre /api/v1/auth/** completo. Sólo son públicos:

```
.requestMatchers(  
    HttpMethod.POST,  
    "/api/v1/auth/registro",  
    "/api/v1/auth/login",  
    "/api/v1/auth/refresh")  
.permitAll()
```

Mejora: logout queda autenticado y un endpoint futuro bajo /auth no se hace público accidentalmente.

Observable: login/registro/refresh funcionan sin credencial previa; logout no.

Criterio de cierre

Al cerrar el punto, login y refresh deben emitir el contrato de tokens esperado y el perfil de producción no debe contener un secreto de fallback.

Paso 6.6.9 - Arrancar y probar el login

Objetivo

Completar **arrancar y probar el login** y comprobar su efecto antes de continuar. Pasamos de los experimentos a una implementación reutilizable con JWT 0.13.0, claves externalizadas, login, refresh y revocación controlada.

Procedimiento

```
curl -i -X POST http://localhost:8080/api/v1/auth/login \  
-H "Content-Type: application/json" \  
-d '{"username": "ana", "password": "ana123"}'
```

Esperado: 200 con access/refresh/Bearer/expiración.

Prueba contraseña incorrecta: **401 CREDENCIALES_INVALIDAS**, no 500 y sin indicar si el username existe.

Criterio de cierre

Al cerrar el punto, login y refresh deben emitir el contrato de tokens esperado y el perfil de producción no debe contener un secreto de fallback.

Paso 6.6.10 - Decodificar el token

Objetivo

Completar **decodificar el token** y comprobar su efecto antes de continuar. Pasamos de los experimentos a una implementación reutilizable con JWT 0.13.0, claves externalizadas, login, refresh

y revocación controlada.

Procedimiento

Usa el decoder local de 6.5 o jwt.io **sólo con tokens didácticos/locales**. El access debe mostrar sub, jti, tipo=access, id, email, roles, iat y exp.

Pregunta: ¿qué claims son registrados (sub, jti, iat, exp) y cuáles son privados (tipo, id, email, roles)?

Regla: visualizar no equivale a verificar.

Criterio de cierre

Al cerrar el punto, login y refresh deben emitir el contrato de tokens esperado y el perfil de producción no debe contener un secreto de fallback.

Paso 6.6.11 - Probar el refresh

Objetivo

Completar **probar el refresh** y comprobar su efecto antes de continuar. Pasamos de los experimentos a una implementación reutilizable con JWT 0.13.0, claves externalizadas, login, refresh y revocación controlada.

Procedimiento

```
curl -i -X POST http://localhost:8080/api/v1/auth/refresh \
  -H "Content-Type: application/json" \
  -d '{"refresh_token":"<REFRESH>"}'
```

Primer uso: 200 + nuevo par. Segundo uso del mismo refresh: **401 TOKEN_INVALIDO**.

Prueba también un access token en /refresh: debe ser 401 por tipo incorrecto.

Observable: la rotación está demostrada, no sólo documentada.

Criterio de cierre

Al cerrar el punto, login y refresh deben emitir el contrato de tokens esperado y el perfil de producción no debe contener un secreto de fallback.

Paso 6.6.12 - Errores comunes del ejercicio

Objetivo

Completar **errores comunes del ejercicio** y comprobar su efecto antes de continuar. Pasamos de los experimentos a una implementación reutilizable con JWT 0.13.0, claves externalizadas, login, refresh y revocación controlada.

Procedimiento

Error	Observable	Corrección
clave corta/Base64 inválido credenciales malas producen 500	arranque falla excepción sin traducir	secreto Base64 >=256 bits CredencialesInvalidasException → 401

Error	Observable	Corrección
login devuelve 401 antes de autenticar	matcher cerrado	abrir sólo login/registro/refresh
expires_in enorme	ms enviados como s	dividir por 1000
access usado como refresh	tipo no validado	esRefreshTokenValido
refresh reutilizable	sin registro de uso	consumir jti
claims leídos sin firma	decoder usado para seguridad	parseSignedClaims
secreto en repo prod	configuración insegura	\${JWT_SECRET}

Materialización exacta del contrato de error en el proyecto final

El tratamiento de errores se materializa en tres piezas concretas:

- `CredencialesInvalidasException.java`: representa el rechazo de usuario/contraseña con código estable `CREDENCIALES_INVALIDAS`;
- `TokenInvalidoException.java`: representa JWT expirado, revocado, mal firmado o de tipo incorrecto con código `TOKEN_INVALIDO`;
- `AuthExceptionHandler.java`: traduce ambas excepciones a HTTP 401 usando el mismo `ErrorResponse` heredado de M5, incluido `traceId`.

Criterio de cierre

Al cerrar el punto, login y refresh deben emitir el contrato de tokens esperado y el perfil de producción no debe contener un secreto de fallback.

Paso 6.6.13 - Reto resuelto: endpoint para logout

Objetivo

Completar **reto resuelto: endpoint para logout** y comprobar su efecto antes de continuar. Pasamos de los experimentos a una implementación reutilizable con JJWT 0.13.0, claves externalizadas, login, refresh y revocación controlada.

Procedimiento

`TokenRevocationService` guarda `jti` → exp de access tokens revocados. Logout:

```
curl -i -X POST \
  -H "Authorization: Bearer $TOKEN" \
  http://localhost:8080/api/v1/auth/logout
```

Esperado: 204 No Content.

El filtro 6.7 consultará esa revocación; después del logout, reutilizar el token debe producir 401.

Limitación explicada: el mapa en memoria no sirve como solución distribuida; cada instancia tendría una blacklist distinta. Producción requiere almacenamiento compartido o una política equivalente.

Cierre 6.6

JJWT operativo, secretos coherentes, tipos access/refresh, refresh rotativo y logout diseñado. El siguiente punto activa el consumo de access tokens en cada petición.

6.7 — Filtro JWT

Criterio de cierre

Al cerrar el punto, login y refresh deben emitir el contrato de tokens esperado y el perfil de producción no debe contener un secreto de fallback.

Práctica 6.7 - Filtro JWT y reconstrucción de la identidad

El Bearer token debe convertirse en una identidad de Spring Security antes de evaluar permisos. El filtro autentica; no decide políticas de autorización.

Paso 6.7.1 - Verificar JwtService y UsuarioPrincipal

Objetivo

Completar **verificar jwtservice y usuarioprincipal** y comprobar su efecto antes de continuar. El Bearer token debe convertirse en una identidad de Spring Security antes de evaluar permisos. El filtro autentica; no decide políticas de autorización.

Procedimiento

Comprueba que JwtService expone extraerClaims, extraerAuthorities y validación de access; UsuarioPrincipal debe aceptar (Long id, String username, String email, Collection<GrantedAuthority>).

```
grep -R "class JwtService|class UsuarioPrincipal" src/main/java/es/mecd/demo/miproyecto/auth
```

Por qué: el filtro no consulta DB; construye principal desde claims ya verificados.

Criterio de cierre

Al cerrar el punto, un Bearer token válido debe poblar el SecurityContext y una cabecera ausente no debe romper rutas públicas.

Paso 6.7.2 - Crear JwtAuthenticationFilter

Objetivo

Completar **crear jwtauthenticationfilter** y comprobar su efecto antes de continuar. El Bearer token debe convertirse en una identidad de Spring Security antes de evaluar permisos. El filtro autentica; no decide políticas de autorización.

Procedimiento

Archivo: auth/JwtAuthenticationFilter.java.

El filtro:

1. lee Authorization;
2. si no empieza por Bearer, continúa;
3. exige access token válido;
4. comprueba jti no revocado;
5. lee id/username/email/roles;
6. construye UsuarioPrincipal;
7. crea UsernamePasswordAuthenticationToken con authorities;
8. lo coloca en SecurityContextHolder;

9. siempre continúa la cadena.

No imprime el token completo en logs.

Error/corrección: olvidar `filterChain.doFilter` deja la petición bloqueada.

Referencia de implementación

```
package es.mecd.demo.miproyecto.auth;

import io.jsonwebtoken.Claims;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.List;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

/** Autentica una petición a partir de Authorization: Bearer. */
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {
    private static final Logger LOG = LoggerFactory.getLogger(JwtAuthenticationFilter.class);

    private final JwtService jwtService;
    private final TokenRevocationService tokenRevocationService;

    public JwtAuthenticationFilter(
        JwtService jwtService,
        TokenRevocationService tokenRevocationService) {
        this.jwtService = jwtService;
        this.tokenRevocationService = tokenRevocationService;
    }

    @Override
    protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain filterChain) throws ServletException, IOException {
        String authHeader = request.getHeader("Authorization");
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        String token = authHeader.substring(7);
        if (!jwtService.esAccessTokenValido(token)) {
            LOG.debug("JWT inválido o no-access en {}", request.getRequestURI());
            filterChain.doFilter(request, response);
            return;
        }
    }
}
```

```

    }

    try {
        Claims claims = jwtService.extraerClaims(token);
        if (claims.getId() == null
            || tokenRevocationService.isAccessRevoked(claims.getId())) {
            filterChain.doFilter(request, response);
            return;
        }

        Number rawId = claims.get("id", Number.class);
        Long id = rawId == null ? null : rawId.longValue();
        String username = claims.getSubject();
        String email = claims.get("email", String.class);
        List<GrantedAuthority> authorities = jwtService.extraerAuthorities(token);

        UsuarioPrincipal principal = new UsuarioPrincipal(
            id, username, email, authorities);
        UsernamePasswordAuthenticationToken authentication =
            new UsernamePasswordAuthenticationToken(
                principal, null, authorities);
        authentication.setDetails(
            new WebAuthenticationDetailsSource().buildDetails(request));
        SecurityContextHolder.getContext().setAuthentication(authentication);
        LOG.debug("Usuario {} autenticado en {}", username, request.getRequestURI());
    } catch (RuntimeException ex) {
        SecurityContextHolder.clearContext();
        LOG.debug(
            "No se pudo construir el principal en {}: {}",
            request.getRequestURI(), ex.getMessage());
    }

    filterChain.doFilter(request, response);
}
}

```

Criterio de cierre

Al cerrar el punto, un Bearer token válido debe poblar el SecurityContext y una cabecera ausente no debe romper rutas públicas.

Paso 6.7.3 - Registrar el filtro en la cadena

Objetivo

Completar **registrar el filtro en la cadena** y comprobar su efecto antes de continuar. El Bearer token debe convertirse en una identidad de Spring Security antes de evaluar permisos. El filtro autentica; no decide políticas de autorización.

Procedimiento

```

.sessionManagement(session -> session
    .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)

```

Observable: no se crea sesión HTTP y el JWT tiene oportunidad de autenticar antes de autorización.

Criterio de cierre

Al cerrar el punto, un Bearer token válido debe poblar el SecurityContext y una cabecera ausente no debe romper rutas públicas.

Paso 6.7.4 - Desactivar HTTP Basic

Objetivo

Completar **desactivar http basic** y comprobar su efecto antes de continuar. El Bearer token debe convertirse en una identidad de Spring Security antes de evaluar permisos. El filtro autentica; no decide políticas de autorización.

Procedimiento

Se elimina definitivamente:

```
.httpBasic(Customizer.withDefaults())
```

Cierre de HTTP Basic. Desde aquí, usuario/contraseña sólo se envía al login; el resto usa Bearer.

Comprobación negativa: SecurityConfig final no puede contener `.httpBasic()`.

Criterio de cierre

Al cerrar el punto, un Bearer token válido debe poblar el SecurityContext y una cabecera ausente no debe romper rutas públicas.

Paso 6.7.5 - Arrancar y probar

Objetivo

Completar **arrancar y probar** y comprobar su efecto antes de continuar. El Bearer token debe convertirse en una identidad de Spring Security antes de evaluar permisos. El filtro autentica; no decide políticas de autorización.

Procedimiento

```
TOKEN=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"ana","password":"ana123"}' | jq -r '.access_token')
```

```
curl -i http://localhost:8080/api/v1/perfil
curl -i -H "Authorization: Bearer $TOKEN" http://localhost:8080/api/v1/perfil
curl -i http://localhost:8080/api/v1/public/info
```

Esperado: **401 / 200 / 200.**

Criterio de cierre

Al cerrar el punto, un Bearer token válido debe poblar el SecurityContext y una cabecera ausente no debe romper rutas públicas.

Paso 6.7.6 - Verificar UsuarioPrincipal

Objetivo

Completar **verificar usuarioprincipal** y comprobar su efecto antes de continuar. El Bearer token debe convertirse en una identidad de Spring Security antes de evaluar permisos. El filtro autentica; no decide políticas de autorización.

Procedimiento

PerfilController pasa a recibir:

```
@AuthenticationPrincipal UsuarioPrincipal user
```

y devuelve id, username, email y authorities.

Activación del principal JWT. El principal preparado en 6.4 ya está activo para peticiones JWT.

Criterio de cierre

Al cerrar el punto, un Bearer token válido debe poblar el SecurityContext y una cabecera ausente no debe romper rutas públicas.

Paso 6.7.7 - Probar con rol ADMIN

Objetivo

Completar **probar con rol admin** y comprobar su efecto antes de continuar. El Bearer token debe convertirse en una identidad de Spring Security antes de evaluar permisos. El filtro autentica; no decide políticas de autorización.

Procedimiento

Login como admin + /api/v1/admin/usuarios → 200. Login como ana → 403.

```
curl -i -H "Authorization: Bearer $TOKEN_ADMIN" \  
http://localhost:8080/api/v1/admin/usuarios
```

Diferencia: 403 significa que el filtro autenticó correctamente, pero authorities no satisfacen ADMIN.

Criterio de cierre

Al cerrar el punto, un Bearer token válido debe poblar el SecurityContext y una cabecera ausente no debe romper rutas públicas.

Paso 6.7.8 - Probar con un token inválido

Objetivo

Completar **probar con un token inválido** y comprobar su efecto antes de continuar. El Bearer token debe convertirse en una identidad de Spring Security antes de evaluar permisos. El filtro autentica; no decide políticas de autorización.

Procedimiento

```
curl -i -H "Authorization: Bearer tokenInvalido" \  
http://localhost:8080/api/v1/perfil
```

Esperado: 401. El filtro no convierte el token inválido en 500; continúa sin identidad y la cadena rechaza el recurso protegido.

Criterio de cierre

Al cerrar el punto, un Bearer token válido debe poblar el SecurityContext y una cabecera ausente no debe romper rutas públicas.

Paso 6.7.9 - Probar con cabecera mal formada

Objetivo

Completar **probar con cabecera mal formada** y comprobar su efecto antes de continuar. El Bearer token debe convertirse en una identidad de Spring Security antes de evaluar permisos. El filtro autentica; no decide políticas de autorización.

Procedimiento

```
curl -i -H "Authorization: $TOKEN" http://localhost:8080/api/v1/perfil
```

Esperado: 401. Bearer es parte del contrato y evita interpretar otros esquemas como JWT.

Criterio de cierre

Al cerrar el punto, un Bearer token válido debe poblar el SecurityContext y una cabecera ausente no debe romper rutas públicas.

Paso 6.7.10 - Depurar el filtro

Objetivo

Completar **depurar el filtro** y comprobar su efecto antes de continuar. El Bearer token debe convertirse en una identidad de Spring Security antes de evaluar permisos. El filtro autentica; no decide políticas de autorización.

Procedimiento

Dev ya registra el paquete de la aplicación en DEBUG. Añade temporalmente si lo necesitas:

```
logging.level.org.springframework.security=DEBUG
```

Observa ruta, usuario y decisiones; **no** copies tokens completos al log. Restaura el nivel global tras depurar.

Criterio de cierre

Al cerrar el punto, un Bearer token válido debe poblar el SecurityContext y una cabecera ausente no debe romper rutas públicas.

Paso 6.7.11 - Errores comunes del ejercicio

Objetivo

Completar **errores comunes del ejercicio** y comprobar su efecto antes de continuar. El Bearer token debe convertirse en una identidad de Spring Security antes de evaluar permisos. El filtro autentica; no decide políticas de autorización.

Procedimiento

Error	Causa	Corrección
401 siempre	filtro no registrado	addFilterBefore
403 con token correcto	authorities vacías/incorrectas	construir Authentication con roles
petición colgada	falta doFilter	continuar cadena
500 token inválido	parseo no controlado	esAccessTokenValido + catch
públicos devuelven 401	filtro rechaza ausencia	continuar sin autenticar
sesión aparece	falta STATELESS	fijar policy
refresh autentica recursos	no se distingue tipo	sólo tipo=access
token de logout sigue activo	blacklist ignorada	consultar jti revocado

Criterio de cierre

Al cerrar el punto, un Bearer token válido debe poblar el SecurityContext y una cabecera ausente no debe romper rutas públicas.

Paso 6.7.12 - Reto resuelto: SpEL con ID del usuario**Objetivo**

Completar **reto resuelto: spel con id del usuario** y comprobar su efecto antes de continuar. El Bearer token debe convertirse en una identidad de Spring Security antes de evaluar permisos. El filtro autentica; no decide políticas de autorización.

Procedimiento

Endpoint:

```
@GetMapping("/{id}")
@PreAuthorize("#id == authentication.principal.id or hasRole('ADMIN')")
public UsuarioResponseDTO perfilPorId(@PathVariable Long id) {
    return userService.consultar(id);
}
```

Prueba:

- Ana consulta su ID → 200.
- Ana consulta ID ajeno → 403.
- Admin consulta cualquier ID → 200 **con el usuario solicitado**.

Decisión técnica: devolver los claims del principal cuando un administrador consulta otro id produciría un resultado incorrecto; el controlador autoriza primero y devuelve el usuario realmente solicitado.

Cierre 6.7

Filtro activo, STATELESS, HTTP Basic cerrado y UsuarioPrincipal integrado. Sólo queda consolidar errores y el flujo completo.

6.8 — Protección de endpoints con JWT**Criterio de cierre**

Al cerrar el punto, un Bearer token válido debe poblar el SecurityContext y una cabecera ausente no debe romper rutas públicas.

Práctica 6.8 - Configuración final stateless y manejo de errores

Cerramos la arquitectura stateless: allowlist pública explícita, roles, manejadores 401/403, CORS y logout/revocación coherentes.

Paso 6.8.1 - Crear JwtAuthenticationEntryPoint

Objetivo

Completar **crear jwtauthenticationentrypoint** y comprobar su efecto antes de continuar. Cerramos la arquitectura stateless: allowlist pública explícita, roles, manejadores 401/403, CORS y logout/revocación coherentes.

Procedimiento

Implementa AuthenticationEntryPoint y delega en SecurityErrorWriter.

Esperado 401: JSON ErrorResponse con timestamp, status, codigo=NO_AUTENTICADO, mensaje, path y traceId.

Por qué no @RestControllerAdvice: la petición puede fallar dentro de Security antes de entrar en MVC.

Referencia de implementación

```
package es.mecd.demo.miproyecto.auth;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import org.springframework.security.core.AuthenticationException;
import org.springframework.security.web.AuthenticationEntryPoint;
import org.springframework.stereotype.Component;

/** Respuesta JSON uniforme cuando falta autenticación válida. */
@Component
public class JwtAuthenticationEntryPoint implements AuthenticationEntryPoint {
    private final SecurityErrorWriter errorWriter;

    public JwtAuthenticationEntryPoint(SecurityErrorWriter errorWriter) {
        this.errorWriter = errorWriter;
    }

    @Override
    public void commence(
        HttpServletRequest request,
        HttpServletResponse response,
        AuthenticationException authException) throws IOException {
        errorWriter.write(
            request,
            response,
            HttpServletResponse.SC_UNAUTHORIZED,
            "NO_AUTENTICADO",
            "Se requiere autenticación para acceder a este recurso");
    }
}
```

```

    }
}

```

Criterio de cierre

Al cerrar el punto, la API debe ser STATELESS, sin HTTP Basic, con 401/403 JSON uniformes y reglas de rol verificables.

Paso 6.8.2 - Crear JwtAccessDeniedHandler

Objetivo

Completar **crear jwtaccessdeniedhandler** y comprobar su efecto antes de continuar. Cerramos la arquitectura stateless: allowlist pública explícita, roles, manejadores 401/403, CORS y logout/revocación coherentes.

Procedimiento

Implementa AccessDeniedHandler y devuelve el mismo contrato con status 403 y código=ACCESO_DENEGADO.

Observable: 401 y 403 ya no dependen de páginas HTML ni de formatos distintos al resto de M5.

Referencia de implementación

```

package es.mecd.demo.miproyecto.auth;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.web.access.AccessDeniedHandler;
import org.springframework.stereotype.Component;

/** Respuesta JSON uniforme para un usuario autenticado sin permisos. */
@Component
public class JwtAccessDeniedHandler implements AccessDeniedHandler {
    private final SecurityErrorWriter errorWriter;

    public JwtAccessDeniedHandler(SecurityErrorWriter errorWriter) {
        this.errorWriter = errorWriter;
    }

    @Override
    public void handle(
        HttpServletRequest request,
        HttpServletResponse response,
        AccessDeniedException accessDeniedException) throws IOException {
        errorWriter.write(
            request,
            response,
            HttpServletResponse.SC_FORBIDDEN,
            "ACCESO_DENEGADO",
            "No tiene permisos para acceder a este recurso");
    }
}

```

Criterio de cierre

Al cerrar el punto, la API debe ser STATELESS, sin HTTP Basic, con 401/403 JSON uniformes y reglas de rol verificables.

Paso 6.8.3 - Actualizar SecurityConfig

Objetivo

Completar **actualizar securityconfig** y comprobar su efecto antes de continuar. Cerramos la arquitectura stateless: allowlist pública explícita, roles, manejadores 401/403, CORS y logout/revocación coherentes.

Procedimiento

Cadena final:

```
.csrf(AbstractHttpConfigurer::disable)
.cors(Customizer.withDefaults())
.sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
.authorizeHttpRequests(...)
.exceptionHandling(ex -> ex
    .authenticationEntryPoint(entryPoint)
    .accessDeniedHandler(deniedHandler))
.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
```

Reglas públicas específicas, ADMIN, GESTOR/ADMIN y `anyRequest().authenticated()` al final.

Regresión M5: CORS y OpenAPI se conservan explícitamente.

Criterio de cierre

Al cerrar el punto, la API debe ser STATELESS, sin HTTP Basic, con 401/403 JSON uniformes y reglas de rol verificables.

Paso 6.8.4 - Añadir endpoint de gestor

Objetivo

Completar **añadir endpoint de gestor** y comprobar su efecto antes de continuar. Cerramos la arquitectura stateless: allowlist pública explícita, roles, manejadores 401/403, CORS y logout/revocación coherentes.

Procedimiento

GestorController expone:

GET /api/v1/gestor/documentos

Y SecurityConfig exige GESTOR o ADMIN. El endpoint existe desde el reto 6.2 y aquí se verifica bajo JWT, no se duplica.

Criterio de cierre

Al cerrar el punto, la API debe ser STATELESS, sin HTTP Basic, con 401/403 JSON uniformes y reglas de rol verificables.

Paso 6.8.5 - Añadir rol GESTOR al inicializador

Objetivo

Completar **añadir rol gestor al inicializador** y comprobar su efecto antes de continuar. Cerramos la arquitectura stateless: allowlist pública explícita, roles, manejadores 401/403, CORS y logout/revocación coherentes.

Procedimiento

UsuariosInicialesConfig ya es idempotente y conserva:

gestor / gestor123 → USER + GESTOR

Continuidad: migrar a DB/JWT no puede destruir un reto de puntos anteriores.

Criterio de cierre

Al cerrar el punto, la API debe ser STATELESS, sin HTTP Basic, con 401/403 JSON uniformes y reglas de rol verificables.

Paso 6.8.6 - Probar el flujo completo

Objetivo

Completar **probar el flujo completo** y comprobar su efecto antes de continuar. Cerramos la arquitectura stateless: allowlist pública explícita, roles, manejadores 401/403, CORS y logout/revocación coherentes.

Procedimiento

Gestor:

1. login → token;
2. perfil → 200;
3. /gestor/documentos → 200;
4. /admin/usuarios → 403.

Esto distingue identidad válida de permiso insuficiente.

Criterio de cierre

Al cerrar el punto, la API debe ser STATELESS, sin HTTP Basic, con 401/403 JSON uniformes y reglas de rol verificables.

Paso 6.8.7 - Probar sin token

Objetivo

Completar **probar sin token** y comprobar su efecto antes de continuar. Cerramos la arquitectura stateless: allowlist pública explícita, roles, manejadores 401/403, CORS y logout/revocación coherentes.

Procedimiento

```
curl -i http://localhost:8080/api/v1/perfil
curl -i http://localhost:8080/api/v1/public/info
```

Esperado: 401 JSON con traceId / 200.

Pregunta: ¿el 401 mantiene el contrato de errores M5? Debe hacerlo.

Criterio de cierre

Al cerrar el punto, la API debe ser STATELESS, sin HTTP Basic, con 401/403 JSON uniformes y reglas de rol verificables.

Paso 6.8.8 - Probar refresh token

Objetivo

Completar **probar refresh token** y comprobar su efecto antes de continuar. Cerramos la arquitectura stateless: allowlist pública explícita, roles, manejadores 401/403, CORS y logout/revocación coherentes.

Procedimiento

Obtén refresh, úsalo una vez y comprueba que devuelve un par nuevo. Reutiliza el antiguo: 401.

Decisión técnica: la rotación está implementada, no se limita a emitir otro token mientras el anterior sigue válido.

Criterio de cierre

Al cerrar el punto, la API debe ser STATELESS, sin HTTP Basic, con 401/403 JSON uniformes y reglas de rol verificables.

Paso 6.8.9 - Probar logout

Objetivo

Completar **probar logout** y comprobar su efecto antes de continuar. Cerramos la arquitectura stateless: allowlist pública explícita, roles, manejadores 401/403, CORS y logout/revocación coherentes.

Procedimiento

Login → perfil 200 → logout 204 → mismo access token en perfil → 401.

El filtro consulta jti revocado antes de construir identidad.

Criterio de cierre

Al cerrar el punto, la API debe ser STATELESS, sin HTTP Basic, con 401/403 JSON uniformes y reglas de rol verificables.

Paso 6.8.10 - Escribir tests de integración

Objetivo

Completar **escribir tests de integración** y comprobar su efecto antes de continuar. Cerramos la arquitectura stateless: allowlist pública explícita, roles, manejadores 401/403, CORS y logout/revocación coherentes.

Procedimiento

Incluye tests reales del flujo, además de tests por rol. Cuando un test necesita un mock Spring usa `@MockitoBean`; para el filtro y la base de datos se prefiere `@SpringBootTest`.

Ejecutar:

```
./mvnw -B test
```

Criterio de cierre

Al cerrar el punto, la API debe ser STATELESS, sin HTTP Basic, con 401/403 JSON uniformes y reglas de rol verificables.

Paso 6.8.11 - Errores comunes del ejercicio

Objetivo

Completar **errores comunes del ejercicio** y comprobar su efecto antes de continuar. Cerramos la arquitectura stateless: allowlist pública explícita, roles, manejadores 401/403, CORS y logout/revocación coherentes.

Procedimiento

Error	Causa	Solución
401 HTML 403 no uniforme gestor 403 refresh viejo funciona dos veces	entrypoint ausente handler ausente rol no emitido/asignado sin consumo de jti	configurar handler JSON JwtAccessDeniedHandler revisar initializer/claims rotación real
logout no invalida CORS falla tras Security OpenAPI 401	filtro no consulta revocación .cors() omitido las rutas públicas dejaron de estar permitidas	blacklist por jti integrar CORS M5 rutas Swagger/docs públicas
anyRequest tapa públicos	orden incorrecto	regla general al final

Criterio de cierre

Al cerrar el punto, la API debe ser STATELESS, sin HTTP Basic, con 401/403 JSON uniformes y reglas de rol verificables.

Paso 6.8.12 - Reto resuelto: consultar el propio usuario

Objetivo

Completar **reto resuelto: consultar el propio usuario** y comprobar su efecto antes de continuar. Cerramos la arquitectura stateless: allowlist pública explícita, roles, manejadores 401/403, CORS y logout/revocación coherentes.

Procedimiento

GET /api/v1/perfil/usuario

Devuelve id, username y email desde UsuarioPrincipal, sin acceso a repositorio.

```
curl -H "Authorization: Bearer $TOKEN" \
http://localhost:8080/api/v1/perfil/usuario
```

Observable: datos coinciden con claims verificados.

Cierre 6.8

Cadena final, errores uniformes, roles, refresh/logout y rutas públicas/protegidas integrados. 6.9 convierte todo esto en regresión automática.

6.9 — Testing de seguridad

Criterio de cierre

Al cerrar el punto, la API debe ser STATELESS, sin HTTP Basic, con 401/403 JSON uniformes y reglas de rol verificables.

Práctica 6.9 - Pruebas de seguridad

La seguridad queda demostrada con pruebas reproducibles. Se cubren caminos positivos y negativos, flujo JWT real y regresiones de los controladores heredados.

Paso 6.9.1 - Añadir spring-security-test

Objetivo

Completar **añadir spring-security-test** y comprobar su efecto antes de continuar. La seguridad queda demostrada con pruebas reproducibles. Se cubren caminos positivos y negativos, flujo JWT real y regresiones de los controladores heredados.

Procedimiento

La dependencia ya está en el POM con scope=test.

```
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-test</artifactId>
  <scope>test</scope>
</dependency>
```

Observable: están disponibles @WithMockUser, @WithUserDetails y post-processors.

Criterio de cierre

Al cerrar el punto, la suite debe demostrar login, refresh, logout, filtro JWT, rutas públicas y autorización por roles.

Paso 6.9.2 - Crear test del UsuarioController

Objetivo

Completar **crear test del usuariocontroller** y comprobar su efecto antes de continuar. La seguridad queda demostrada con pruebas reproducibles. Se cubren caminos positivos y negativos, flujo JWT real y regresiones de los controladores heredados.

Procedimiento

UsuarioControllerSecurityTest usa contexto completo de test para validar la configuración real y cubre:

- anónimo → 401 + NO_AUTENTICADO;
- USER → 403 + ACCESO_DENEGADO;
- ADMIN → 200.

M6 evita @MockBean. Cuando se necesita sustituir un bean Spring, el contrato exige @MockitoBean.

Criterio de cierre

Al cerrar el punto, la suite debe demostrar login, refresh, logout, filtro JWT, rutas públicas y autorización por roles.

Paso 6.9.3 - Ejecutar los tests

Objetivo

Completar **ejecutar los tests** y comprobar su efecto antes de continuar. La seguridad queda demostrada con pruebas reproducibles. Se cubren caminos positivos y negativos, flujo JWT real y regresiones de los controladores heredados.

Procedimiento

Ejecutamos la suite **acumulativa completa**, no sólo los tests nuevos:

```
./mvnw -B clean verify
```

El objetivo final es **58 tests**: los 32 heredados de M5 más 26 tests M6, sin fallos, errores ni skipped, y con Checkstyle a cero.

Cinco @WebMvcTest heredados se mantienen como tests de controlador/error/CORS, no como tests de seguridad. Al entrar Spring Security en el classpath de M6 se evolucionan con @AutoConfigureMockMvc(addFilters = false) para que sigan verificando su contrato M5 y, además, con @MockitoBean para los dos colaboradores del JwtAuthenticationFilter (JwtService y TokenRevocationService). Esto último es necesario porque desactivar la aplicación de filtros en MockMvc **no desactiva la creación del bean filtro** durante el arranque del slice. La seguridad se prueba por separado en los tests dedicados de este módulo y en la prueba de ejecución real.

Los cinco ficheros heredados que reciben esa modificación mínima y exclusivamente de aislamiento del slice son:

- AlumnoControllerTest.java;
- ErrorHandlingTest.java;
- CorsConfigTest.java;
- ExpedienteControllerTest.java;
- FicheroExceptionHandlerTest.java.

En cada uno de esos cinco slices heredados, el patrón de aislamiento queda explícito:

```
@AutoConfigureMockMvc(addFilters = false)
```

```
@MockitoBean
```

```
private JwtService jwtService;
```

```
@MockitoBean
```

```
private TokenRevocationService tokenRevocationService;
```

Si cambia una regla y abre /admin, los escenarios USER/anónimo de la suite M6 deben fallar inmediatamente.

Criterio de cierre

Al cerrar el punto, la suite debe demostrar login, refresh, logout, filtro JWT, rutas públicas y autorización por roles.

Paso 6.9.4 - Testear endpoint de gestor

Objetivo

Completar **testear endpoint de gestor** y comprobar su efecto antes de continuar. La seguridad queda demostrada con pruebas reproducibles. Se cubren caminos positivos y negativos, flujo JWT real y regresiones de los controladores heredados.

Procedimiento

GestorSecurityTest verifica:

GESTOR → 200
ADMIN → 200
USER → 403

Pregunta: USER está autenticado, por eso el resultado correcto es 403 y no 401.

Criterio de cierre

Al cerrar el punto, la suite debe demostrar login, refresh, logout, filtro JWT, rutas públicas y autorización por roles.

Paso 6.9.5 - Testear endpoint público

Objetivo

Completar **testear endpoint público** y comprobar su efecto antes de continuar. La seguridad queda demostrada con pruebas reproducibles. Se cubren caminos positivos y negativos, flujo JWT real y regresiones de los controladores heredados.

Procedimiento

PublicControllerSecurityTest llama /api/v1/public/info sin autenticación y exige 200.

Este test protege contra una regresión frecuente: que un cambio en la cadena convierta accidentalmente un endpoint público en privado.

Criterio de cierre

Al cerrar el punto, la suite debe demostrar login, refresh, logout, filtro JWT, rutas públicas y autorización por roles.

Paso 6.9.6 - Testear login

Objetivo

Completar **testear login** y comprobar su efecto antes de continuar. La seguridad queda demostrada con pruebas reproducibles. Se cubren caminos positivos y negativos, flujo JWT real y regresiones de los controladores heredados.

Procedimiento

La suite end-to-end AuthFlowIntegrationTest ejecuta login real con ana/ana123 y exige token_type=Bearer.

La documentación conserva también el enfoque de controller-slice con servicio mockeado, pero el proyecto final prioriza una prueba real del flujo de seguridad.

Credenciales inválidas deben producir 401 CREDENCIALES_INVALIDAS, no 409 del NegocioException histórico.

Criterio de cierre

Al cerrar el punto, la suite debe demostrar login, refresh, logout, filtro JWT, rutas públicas y autorización por roles.

Paso 6.9.7 - Testear refresh

Objetivo

Completar **testear refresh** y comprobar su efecto antes de continuar. La seguridad queda demostrada con pruebas reproducibles. Se cubren caminos positivos y negativos, flujo JWT real y regresiones de los controladores heredados.

Procedimiento

Tras login, el test usa el refresh válido → 200. Vuelve a usar **el mismo** → 401 TOKEN_INVALIDO.

Así se verifica firma/tipo y la rotación, no sólo el controlador.

Criterio de cierre

Al cerrar el punto, la suite debe demostrar login, refresh, logout, filtro JWT, rutas públicas y autorización por roles.

Paso 6.9.8 - Testear logout

Objetivo

Completar **testear logout** y comprobar su efecto antes de continuar. La seguridad queda demostrada con pruebas reproducibles. Se cubren caminos positivos y negativos, flujo JWT real y regresiones de los controladores heredados.

Procedimiento

El mismo flujo hace logout con Bearer válido → 204 y luego intenta reutilizar access → 401.

Pregunta: 204 es apropiado porque la operación se completó y no necesita body.

Criterio de cierre

Al cerrar el punto, la suite debe demostrar login, refresh, logout, filtro JWT, rutas públicas y autorización por roles.

Paso 6.9.9 - Testear con post-processors

Objetivo

Completar **testear con post-processors** y comprobar su efecto antes de continuar. La seguridad queda demostrada con pruebas reproducibles. Se cubren caminos positivos y negativos, flujo JWT real y regresiones de los controladores heredados.

Procedimiento

UsuarioControllerSecurityTest usa:

```
.with(user("admin").roles("ADMIN"))
```

para aplicar una identidad a una petición concreta, además de @WithMockUser.

Ventaja: permite cambiar identidad por request sin modificar el contexto del método completo.

Criterio de cierre

Al cerrar el punto, la suite debe demostrar login, refresh, logout, filtro JWT, rutas públicas y autorización por roles.

Paso 6.9.10 - Testear con @WithUserDetails

Objetivo

Completar **testear con @withuserdetails** y comprobar su efecto antes de continuar. La seguridad queda demostrada con pruebas reproducibles. Se cubren caminos positivos y negativos, flujo JWT real y regresiones de los controladores heredados.

Procedimiento

UsuarioControllerIntegrationTest usa:

```
@WithUserDetails("admin")
```

con @SpringBootTest. El usuario existe por el inicializador idempotente; Spring lo carga mediante nuestro UsuarioDetailsService real.

Observable: el test cubre la transformación DB → UserDetails → roles.

Criterio de cierre

Al cerrar el punto, la suite debe demostrar login, refresh, logout, filtro JWT, rutas públicas y autorización por roles.

Paso 6.9.11 - Testear el filtro JWT

Objetivo

Completar **testear el filtro jwt** y comprobar su efecto antes de continuar. La seguridad queda demostrada con pruebas reproducibles. Se cubren caminos positivos y negativos, flujo JWT real y regresiones de los controladores heredados.

Procedimiento

JwtServiceIntegrationTest verifica además, contra el contexto Spring real, que access y refresh son tipos mutuamente excluyentes, que el access contiene roles y que exp > iat. Es la evidencia directa de que el contrato criptográfico de 6.6 no es sólo sintáctico.

JwtFilterIntegrationTest genera un access token real con JwtService:

- token válido → perfil 200;
- token inválido → 401;
- sin token → 401.

Por qué no jwt() como sustituto: nuestro sistema no es el resource-server OAuth2 de Spring; necesitamos demostrar que **nuestro filtro** procesa una cabecera firmada por **nuestro JwtService**.

Referencia de implementación

```
package es.mecd.demo.miproyecto.auth;

import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.ActiveProfiles;
import org.springframework.test.web.servlet.MockMvc;

@SpringBootTest
@AutoConfigureMockMvc
@ActiveProfiles("test")
class JwtFilterIntegrationTest {
    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private JwtService jwtService;

    @Autowired
    private UsuarioRepository usuarioRepository;

    @Test
    void perfil_debeDevolver200_conTokenValido() throws Exception {
        Usuario ana = usuarioRepository.findByUsername("ana").orElseThrow();
        String token = jwtService.generarToken(ana);

        mockMvc.perform(get("/api/v1/perfil")
            .header("Authorization", "Bearer " + token))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.username").value("ana"));
    }

    @Test
    void perfil_debeDevolver401_conTokenInvalido() throws Exception {
        mockMvc.perform(get("/api/v1/perfil")
```

```

        .header("Authorization", "Bearer tokenInvalido"))
        .andExpect(status().isUnauthorized())
        .andExpect(jsonPath("$.codigo").value("NO_AUTENTICADO"));
    }

    @Test
    void perfil_debeDevolver401_sinToken() throws Exception {
        mockMvc.perform(get("/api/v1/perfil"))
            .andExpect(status().isUnauthorized());
    }
}

```

Criterio de cierre

Al cerrar el punto, la suite debe demostrar login, refresh, logout, filtro JWT, rutas públicas y autorización por roles.

Paso 6.9.12 - Errores comunes del ejercicio

Objetivo

Completar **errores comunes del ejercicio** y comprobar su efecto antes de continuar. La seguridad queda demostrada con pruebas reproducibles. Se cubren caminos positivos y negativos, flujo JWT real y regresiones de los controladores heredados.

Procedimiento

Error	Causa	Corrección
clases de security test no existen	falta dependencia	spring-security-test
esperaba 403, recibe 401	no hay identidad	preparar usuario antes de roles
esperaba 200, recibe 403	rol incorrecto	revisar ROLE_/matcher
error JSON no coincide	handlers no activos	verificar cadena completa
@WithUserDetails falla	servicio/usuario ausente	@SpringBootTest + datos
filtro no se prueba	se simuló SecurityContext	enviar JWT real en integración
tests lentos innecesarios	todo es @SpringBootTest	usar slice cuando no se necesite mecanismo real
tests dependen de orden	datos compartidos mutables	escenarios independientes

Criterio de cierre

Al cerrar el punto, la suite debe demostrar login, refresh, logout, filtro JWT, rutas públicas y autorización por roles.

Paso 6.9.13 - Reto resuelto: test completo de autorización

Objetivo

Completar **reto resuelto: test completo de autorización** y comprobar su efecto antes de continuar. La seguridad queda demostrada con pruebas reproducibles. Se cubren caminos positivos y negativos, flujo JWT real y regresiones de los controladores heredados.

Procedimiento

UsuarioControllerCompleteSecurityTest cubre cinco escenarios:

1. anónimo → 401 + estructura NO_AUTENTICADO;
2. USER → 403 + ACCESO_DENEGADO;
3. GESTOR → 403;
4. ADMIN → 200;
5. ADMIN → lista con campos de usuario.

```
./mvnw -B -Dtest=UsuarioControllerCompleteSecurityTest test
```

Observable: si se abre accidentalmente el endpoint admin o cambia el formato de errores, el test falla.

Resultado esperado global del módulo

- 112 pasos completos;
- 45 bloques y 135 subpuntos desarrollados;
- usuarios persistentes;
- Spring Security moderno;
- JWT JJWT 0.13.0;
- STATELESS;
- HTTP Basic eliminado;
- roles y método;
- refresh de un solo uso en el proceso;
- logout con revocación de access por jti;
- 401/403 en formato M5;
- CORS y OpenAPI siguen operativos;
- suite nueva + suite heredada sin eliminar tests.

Criterio de cierre

Al cerrar el punto, la suite debe demostrar login, refresh, logout, filtro JWT, rutas públicas y autorización por roles.

Verificación final del módulo

Ejecuta la suite completa con el perfil de test y revisa los informes de Surefire. La aplicación final debe compilar con Java 17, mantener las pruebas heredadas, responder con el formato de error esperado y demostrar el flujo JWT completo. Comprueba además que SecurityConfig usa SessionCreationPolicy.STATELESS, que HTTP Basic ya no está activo, que el secreto de producción procede de JWT_SECRET y que las rutas públicas se enumeran de forma explícita.